Package 'breedtools'

November 9, 2026

Title Design, Analysis and 'Breedbase' Storage of Plant Breeding Trials

Version 0.1.0

Description Designs and analyses plant breeding field trials, from early generations with a single plot per entry to fully replicated trials, and saves the designs to a 'Breedbase' database. Suggests layout parameters from entry counts and known or unspecified field dimensions. Converts 'agricolae' field books and prepares their trial information for storage.
Designs:
augmented row-column designs with replicated checks (Federer, Nair and Raghavarao (1975) <doi:10.2307/2529426>), including unequal row and column groups, incomplete blocks, extra check replication and D-optimal check placement (Piepho and Williams (2016) <doi:10.2134/agronj2016.06.0325>); multi-location augmented partially replicated (p-rep) designs built by search or from alpha and alpha(2) arrays (Williams, Piepho and Whitaker (2011) <doi:10.1002/bimj.201000102>); and resolvable spatial row-column designs optimized for row-column and linear variance efficiency, with latinized options (Williams, John and Whitaker (2006) <doi:10.1111/j.1541-0420.2005.00393.x>). Analyses give analysis of variance tables with degrees of freedom estimable from the observed plots and adjusted entry means with standard errors, from fixed models, mixed models with random blocks or random entries (best linear unbiased predictions), genotype-by-location interaction, spatial P-spline models with 'SpATS' (Rodriguez-Alvarez et al. (2018) <doi:10.1016/j.spasta.2017.10.003>), two-stage weighted analysis of multi-location trials with heterogeneous error variances (Mohring and Piepho (2009) <doi:10.2135/cropsci2009.02.0083>), and residual maximum likelihood fits of row-column, linear variance and autoregressive spatial models. Draws field maps, writes field books, checks designs, and saves designs as trials with their plots into a 'Breedbase' breeding program and folder through the Breeding API ('BrAPI'), and later the phenotypic data of those plots by plot and trait name, first-stage plot values as a trait and second-stage genotype values as a 'Breedbase' analysis. Also loads 'Breedbase' phenotype files, checks residual variance homogeneity across environments with Hartley's Fmax test (Hartley (1950)

<doi:10.1093/biomet/37.3-4.308>), and groups environments by genotype-by-environment interaction (Lin (1982) <doi:10.1007/BF00276251>).

License GPL-3

URL https://github.com/solgenomics/breedtools, https://solgenomics.github.io/breedtools/

BugReports https://github.com/solgenomics/breedtools/issues

Encoding UTF-8

Language en-US

Roxygen list(markdown = TRUE)

RoxygenNote 8.0.0

Depends R (>= 4.1.0)

Imports dplyr,
tidyr,
purrr,
readxl,
leaflet,
htmltools,
tibble,
tools,
graphics,
grDevices,
jsonlite,
stats,
utils

Suggests knitr,
rmarkdown,
lme4,
httr2 (>= 1.1.0),
curl,
askpass,
agricolae,
Matrix,
SpATS,
withr,
testthat (>= 3.0.0)

VignetteBuilder knitr

Config/testthat/edition 3

Contents

breedtools-package 4
add_breedbase_analysis_values 7
add_environment_metadata 8

add_pairwise_environment_metadata . . . 9
add_plot_coordinates . . . 10
add_plot_names . . . 12
analysis_plot_names . . . 13
analyze_augmented_prep_design . . . 14
analyze_augmented_prep_two_stage . . . 17
analyze_augmented_row_column_design . . . 20
analyze_environment_anova . . . 22
analyze_one_environment . . . 23
analyze_resolvable_row_column . . . 24
analyze_single_stage . . . 26
analyze_two_stage . . . 27
as_breedbase_design . . . 30
augmented_prep_concurrence . . . 31
augmented_prep_design . . . 32
augmented_prep_parameters . . . 36
augmented_row_column_capacity . . . 37
augmented_row_column_design . . . 38
augmented_row_column_layouts . . . 41
breedbase_adjusted_trait . . . 43
breedbase_analysis_plots . . . 44
breedbase_check . . . 45
breedbase_lists . . . 46
breedbase_login . . . 47
breedbase_phenotypes . . . 48
breedbase_refresh . . . 49
breedbase_statistics . . . 50
breedbase_trial_plots . . . 51
calculate_environment_stratification_anova . . . 51
check_design . . . 52
check_reset . . . 53
check_result . . . 53
check_section . . . 54
check_setup . . . 55
check_summary . . . 56
compare_first_stage . . . 56
complete_environment_info . . . 57
design_options . . . 58
detect_design . . . 61
environment_summary_by_group . . . 62
find_outliers . . . 63
first_stage_values . . . 64
get_compatible_groups . . . 65
get_maximal_groups . . . 66
guess_column . . . 66
hartley_critical_sim . . . 67
hartley_fmax_test . . . 68
lin_error_mse . . . 68

lin_group_environments . . . 69
lin_ss_ge . . . 70
lin_test_group . . . 71
make_environment_groups . . . 71
map_environment_groups . . . 72
phenotype_template . . . 74
plot_augmented_prep_design . . . 75
plot_augmented_row_column_design . . . 77
plot_resolvable_row_column_design . . . 78
plot_trait . . . 79
prediction_accuracy . . . 80
prepare_breedbase_pheno . . . 81
prepare_breedbase_trial . . . 82
prepare_design_factors . . . 84
prepare_environment_stratification_data . . . 85
print.augmented_prep_analysis . . . 86
print.augmented_prep_two_stage . . . 86
print.augmented_row_column_analysis . . . 87
print.resolvable_row_column_analysis . . . 87
print.single_stage_analysis . . . 88
print.two_stage_analysis . . . 88
read_breedbase_file . . . 89
read_locations . . . 90
remove_outliers . . . 91
resolvable_row_column_design . . . 92
resolvable_row_column_efficiency . . . 95
run_breedbase_environment_stratification . . . 96
run_breedbase_variance_check . . . 97
run_environment_stratification . . . 98
save_breedbase . . . 100
save_breedbase_analysis . . . 103
save_breedbase_coordinates . . . 106
save_breedbase_first_stage . . . 107
save_breedbase_phenotypes . . . 109
second_stage_values . . . 111

Index **113**

`breedtools-package` *breedtools: design, analysis and 'Breedbase' storage of plant breeding trials*

Description

breedtools designs and analyses plant breeding field trials, from early generations with a single plot per entry to fully replicated trials, and saves the designs into a 'Breedbase' database. The tutorial, `vignette("breedtools")`, walks through every design and analysis.

Which design?

Start with `design_options()` when the number of entries is known but the field dimensions, checks or replication are still to be decided.

- One plot per entry, one location: augmented row-column design with replicated checks.
- One plot per entry per location, several locations: augmented partially replicated (p-rep) design, without checks.
- Every entry in every replicate: resolvable (spatial) row-column design.

Augmented row-column designs

Unreplicated entries plus replicated checks in rows, columns and blocks (Federer, Nair and Raghavarao, 1975; Piepho and Williams, 2016), with equal or unequal row and column groups, complete or incomplete blocks of checks, and Latin or D-optimal check placement that keeps checks apart.

- Plan: `augmented_row_column_layouts()`, `augmented_row_column_capacity()`
- Design: `augmented_row_column_design()`
- Map: `plot_augmented_row_column_design()`
- Analysis (fixed, random blocks, random entries, SpATS spatial): `analyze_augmented_row_column_design()`

Augmented p-rep designs

Multi-location trials in which a share of the entries is repeated at each location (Williams, Piepho and Whitaker, 2011), built by search or from alpha and alpha(2) arrays.

- Plan: `augmented_prep_parameters()`
- Design: `augmented_prep_design()`, `augmented_prep_concurrence()`
- Map: `plot_augmented_prep_design()`
- Analysis (with replicates and fixed or random genotype x location): `analyze_augmented_prep_design()`
- Two-stage weighted analysis with block or SpATS spatial stage 1: `analyze_augmented_prep_two_stage()`

Resolvable row-column designs

Fully replicated designs optimized for row-column and spatial (linear variance) efficiency, with latinized options (Williams, John and Whitaker, 2006).

- Design: `resolvable_row_column_design()`
- Efficiency, neighbours, AR criterion: `resolvable_row_column_efficiency()`
- Map: `plot_resolvable_row_column_design()`
- Analysis (RC, LV + RC and AR + RC by REML): `analyze_resolvable_row_column()`

Saving to Breedbase

Designs are saved as trials with their plots into a breeding program and folder through BrAPI, after checking every name; after harvest, the phenotypic data are saved to the same plots.

- Log in: `breedbase_login()`
- Look up names: `breedbase_programs()`, `breedbase_folders()`, `breedbase_locations()`
- Check location and accessions before designing: `breedbase_check()`
- Convert designs generated with agricolae: `as_breedbase_design()`
- Prepare an agricolae trial with its name, location and program: `prepare_breedbase_trial()`
- Save (with a dry run first): `save_breedbase()`
- Phenotypes: `phenotype_template()` (sheet with the plot names), `save_breedbase_phenotypes()` (values by plot and trait name)
- Analysis results: `first_stage_values()` and `save_breedbase_first_stage()` (plot values as a trait of the field plots, trait made by `breedbase_adjusted_trait()`); `second_stage_values()` and `save_breedbase_analysis()` (genotype values as a Breedbase analysis)

Multi-environment trials from Breedbase

- Read files: `read_breedbase_file()`, `prepare_breedbase_pheno()`
- Error variances (Hartley's Fmax): `run_breedbase_variance_check()`, `hartley_fmax_test()`
- Environment groups (Lin, 1982): `run_environment_stratification()`, `lin_group_environments()`
- Maps: `read_locations()`, `map_environment_groups()`

Trait data and two-stage analysis

- Look: `plot_trait()`; outliers: `find_outliers()`, `remove_outliers()`
- Two-stage analysis for any design: `analyze_two_stage()`; single stage: `analyze_single_stage()`
- Phenotypes of trials from Breedbase: `breedbase_phenotypes()`
- Raw versus first-stage values: `compare_first_stage()`

Checks

`check_setup()`, `check_design()`, `check_result()`, `check_summary()`, `prediction_accuracy()`.

Author(s)

Maintainer: Christiano Costa Simoes <ccs263@cornell.edu>

Authors:

- Christiano Costa Simoes <ccs263@cornell.edu>

See Also

Useful links:

- https://github.com/solgenomics/breedtools
- https://solgenomics.github.io/breedtools/
- Report bugs at https://github.com/solgenomics/breedtools/issues

---

`add_breedbase_analysis_values` *Add results to an analysis that is already in Breedbase*

Description

Adds values (for example a second trait, or the same trait with another statistic) to the plots of an analysis saved before, e.g. with `save_breedbase_analysis()`. Breedbase names the plots of an analysis "<analysis_name>_<field plot name>" ("Analysis of: plots") or "<analysis_name>_<accession>" ("Analysis of: accessions"); the values are uploaded to those plots with the phenotype upload of the website (simple spreadsheet), which first verifies the file and then stores it.

Usage

```
add_breedbase_analysis_values(
  data,
  con,
  analysis_name,
  level = c("plot", "accession"),
  traits = NULL,
  plot = "plot_name",
  accession = "accession",
  statistic = NULL,
  estimate = NULL,
  overwrite = FALSE,
  dry_run = FALSE,
  confirm = interactive()
)
```

Arguments

`data`	A data frame with the field plot names (`level = "plot"`) or the accession names (`level = "accession"`) and one numeric column per trait, e.g. from `first_stage_values()` or `second_stage_values()`.
`con`	A connection from `breedbase_login()` (the site's web pages must be open; see `breedbase_login()`).
`analysis_name`	Name of the existing analysis.
`level`	`"plot"` or `"accession"`, as the analysis was saved.

traits	Trait columns (default: all except the name and design columns). Their names are Breedbase trait names, including composed traits such as "dry matter content percentage|Adjusted Means from Spatial Correction using SpATS R" (see breedbase_adjusted_trait()).
plot, accession	Names of the plot-name and accession columns.
statistic	Optional statistic (see breedbase_statistics()). Values are always stored under "trait|statistic" (a composed trait, created if needed), never under the plain trait, as Breedbase does for analyses. By default the statistic the analysis already uses (from its stored traits); give another to keep, e.g., single-stage means next to two-stage means in the same analysis.
estimate	When the analysis has no statistic yet: "blue", "blup" or "spatial", as in save_breedbase_analysis() (by default from the data).
overwrite	Logical. Replace values already stored.
dry_run	Logical. Only verify (Breedbase checks the file), store nothing.
confirm	Logical. Ask before storing (default in interactive sessions).

Details

Values already present for a plot and trait are not replaced unless overwrite = TRUE: the verification lists them and the upload stops.

Value

Invisibly, a list with the analysis name, the number of plots and values sent, and Breedbase's messages.

Examples

```
## Not run:
con <- breedbase_login("https://cassava-test.sgn.cornell.edu", "jdoe", site_username = "sgn")
# dm1: first-stage values of dry matter, one row per field plot
add_breedbase_analysis_values(dm1, con, analysis_name = "FALL_2026_stage1", level = "plot")

## End(Not run)
```

---

add_environment_metadata

Add environment metadata to result table

Description

Add environment metadata to result table

Usage

```
add_environment_metadata(results, env_info, env_col = "environment")
```

Arguments

`results`	Result data frame.
`env_info`	Environment metadata table.
`env_col`	Name of the environment column in `results`.

Value

Joined data frame.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

membership <- data.frame(
  group_id = "Group_1",
  environment = prepared$env_info$environment[1:2]
)
add_environment_metadata(membership, prepared$env_info)
```

`add_pairwise_environment_metadata` *Add environment metadata to pairwise Lin results*

Description

Add environment metadata to pairwise Lin results

Usage

```
add_pairwise_environment_metadata(pairwise, env_info)
```

Arguments

`pairwise`	Pairwise result table.
`env_info`	Environment metadata table.

Value

Pairwise table with metadata.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

lin <- lin_group_environments(prepared$data)
add_pairwise_environment_metadata(lin$pairwise, prepared$env_info)
```

---

add_plot_coordinates *Add GPS coordinates to the plots of a design*

Description

Adds the GPS position of every plot to `design$design`, so that `save_breedbase()` saves it with the plots and Breedbase shows the trial on its field map. Coordinates are WGS84 decimal degrees (as from a phone or GPS). There are three ways to give them:

Usage

```
add_plot_coordinates(
  design,
  coordinates = NULL,
  latitude = NULL,
  longitude = NULL,
  origin = NULL,
  plot_width = NULL,
  plot_length = NULL,
  col_gap = 0,
  row_gap = 0,
  col_bearing = 90,
  row_bearing = 0
)
```

Arguments

`design`	A design from `augmented_row_column_design()`, `augmented_prep_design()`, `resolvable_row_column_design()`, `as_breedbase_design()` or `prepare_breedbase_trial()`.
`coordinates`	Optional table of coordinates, as described above.
`latitude, longitude`	Optional point per plot, as described above.
`origin`	Optional `c(latitude, longitude)` of the outer corner of the plot in row 1, column 1. For a p-rep design with several locations, a list with one such pair per location, named by location.

`plot_width`, `plot_length`
: Plot size in metres: `plot_width` across columns (from column 1 towards column 2), `plot_length` across rows (from row 1 towards row 2). Used with `origin`.

`col_gap`, `row_gap`
: Space in metres between plots of neighbouring columns and rows (alleys). Used with `origin`.

`col_bearing`, `row_bearing`
: Compass direction (degrees clockwise from north) in which column numbers and row numbers increase; they must be at right angles. The default puts column 1 at the west, columns running east, and row 1 at the south, rows running north. Used with `origin`.

Details

- **From the field corner** (`origin`): give the GPS of the outer corner of the plot in row 1, column 1, the plot size and the directions in which rows and columns run. The four corners of every plot are calculated from its row and column.
- **One point per plot** (`latitude`, `longitude`): vectors with one value per row of `design$design`, or the names of columns of the design that hold them.
- **A table** (`coordinates`), e.g. read with `read_breedbase_file()`: one row per plot, matched by `plot_name` (when both have it) or by plot number (`plots`, `plot_number` or `plot`; for p-rep designs also `location`). It holds either `latitude` and `longitude` (or `WGS84_x`, `WGS84_y`) for a point, or the eight corner columns of the Breedbase GPS upload template (`WGS84_bottom_left_x`, `WGS84_bottom_left_y`, ..., `WGS84_top_left_y`, where `x` is the longitude and `y` the latitude) for a polygon.

With `origin`, positions are calculated on a local flat approximation of the Earth, accurate to centimetres over a field. The design needs `row` and `col` columns (designs converted from agricolae need them in the field book).

The design gets `latitude` and `longitude` columns (the centre of the plot), plus the eight corner columns for polygons. They can be edited. `save_breedbase()` sends a polygon when all eight corner columns of a plot are filled, and otherwise a point from `latitude` and `longitude`. Plots without coordinates are saved without them. For a trial that is already in Breedbase, use `save_breedbase_coordinates()`.

Value

The design with the coordinate columns added.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
# Plots of 3 m x 5 m, 1 m alleys between rows, columns running east:
rc <- add_plot_coordinates(rc, origin = c(7.4931, 3.9035),
                           plot_width = 3, plot_length = 5, row_gap = 1)
head(rc$design[, c("plots", "row", "col", "latitude", "longitude")])
```

```
# A field laid out at an angle: columns running north-east.
rc <- add_plot_coordinates(rc, origin = c(7.4931, 3.9035), plot_width = 3,
                           plot_length = 5, col_bearing = 45, row_bearing = 315)

# One GPS point per plot, measured in the field:
pts <- data.frame(plots = rc$design$plots,
                  latitude = 7.4931 + rc$design$row * 5e-5,
                  longitude = 3.9035 + rc$design$col * 3e-5)
rc <- add_plot_coordinates(rc, coordinates = pts)
```

`add_plot_names` *Add Breedbase plot names to a design*

Description

Adds a `plot_name` column to `design$design` with the names the plots will have in Breedbase, so they can be seen, checked with `breedbase_check()` and changed before `save_breedbase()`. The design functions call this when given a `trial_name`. Run it again to make the names for another trial name or pattern, or edit the column by hand: any unique names are accepted. `save_breedbase()`, `phenotype_template()` and `first_stage_values()` use the stored names.

Usage

```
add_plot_names(
  design,
  trial_name = NULL,
  location = NULL,
  plot_name_pattern = "{trial}_{location}_{accession}_{block}_{rep}",
  plot_names = NULL
)
```

Arguments

`design`	A design from `augmented_row_column_design()`, `augmented_prep_design()`, `resolvable_row_column_design()`, `as_breedbase_design()` or `prepare_breedbase_trial()`.
`trial_name`	The Breedbase trial name. For a multi-location p-rep design, the location is added as in `save_breedbase()` (one trial per location). Can be omitted for a prepared Breedbase trial.
`location`	The Breedbase location of the trial, used by the `{location}` field of the pattern. Not needed for p-rep designs (their own locations are used, or give a named vector mapping them to Breedbase locations, as in `save_breedbase()`) or for a prepared Breedbase trial (its recorded location is used).
`plot_name_pattern`	Pattern for the names, as in `save_breedbase()`: fields `{trial}`, `{location}`, `{accession}`, `{block}`, `{rep}`, `{plot}`, `{row}` and `{col}`.
`plot_names`	Optional names given by you instead of a pattern: one per row of `design$design`, or the name of a column holding them.

Value

The design with a `plot_name` column (`NA` for filler plots, which are not saved) and the trial name in `plot_name_trial`.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
rc <- add_plot_names(rc, "AYT_2026", location = "Ibadan")
head(rc$design$plot_name)
# Other names, e.g. with the field position:
rc <- add_plot_names(rc, "AYT_2026_Ibadan", plot_name_pattern = "{trial}_{plot}_r{row}c{col}")
head(rc$design$plot_name)
```

---

analysis_plot_names *Plot and accession names an analysis will have, before saving*

Description

Shows, without contacting Breedbase, the names under which `save_breedbase_analysis()` (or `add_breedbase_analysis_values()`) stores each row of `data`: Breedbase gives every analysis its own plots, named `"<analysis_name>_<field plot name>"` ("Analysis of: plots", with the accession of the field plot) or `"<analysis_name>_<accession>"` ("Analysis of: accessions"). Use it to see what will be saved, and to compare with the plots of a saved analysis (`breedbase_analysis_plots()`).

Usage

```
analysis_plot_names(
  data,
  analysis_name,
  level = c("plot", "accession"),
  plot = "plot_name",
  accession = "accession",
  traits = NULL
)
```

Arguments

`data`	A data frame with one row per field plot (`level = "plot"`, e.g. from `first_stage_values()`) or per accession (`level = "accession"`, e.g. from `second_stage_values()`) and one column per trait.
`analysis_name`	The analysis name, as given to `save_breedbase_analysis()` (special characters are replaced with `"_"` the same way).
`level`	`"plot"` or `"accession"`.
`plot, accession`	Names of the plot-name and accession columns.
`traits`	Trait columns (default: all except the name and design columns).

Value

A tibble with `analysis_plot_name` (the name in the analysis), `plot_name` (the field plot; for accession analyses, the accession), `accession`, and the trait values.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
d <- rc$design
set.seed(1)
d$yield <- 25 + 0.4 * d$row + rnorm(nrow(d), sd = 2)
fit <- analyze_augmented_row_column_design(d, "yield")
values <- first_stage_values(fit, rc, "AYT_2026", trait = "fresh root yield", location = "Ibadan")
head(analysis_plot_names(values, "AYT_2026_stage1", level = "plot"))
```

---

analyze_augmented_prep_design

Analyze an augmented p-rep design

Description

Fits the fixed-effects incomplete-block model: `y = mean + location + block(location) + genotype + error`. For resolvable designs (a `replicate` column with more than one replicate per location) the model is `y = mean + location + replicate(location) + block(replicate, location) + genotype + error`. With `genotype_by_location = "fixed"` (or `TRUE`) a fixed genotype-by-location interaction is added after genotypes; the error is then estimated only from the p-rep entries, as in a per-location analysis with pooled error (Williams, Piepho and Whitaker, 2011, Section 3). With `genotype_by_location = "random"` the interaction is a random effect, as recommended by the paper when locations are a sample of environments, and the model is fitted by REML with 'lme4'. Sums of squares are sequential, in the order above. Degrees of freedom in the formatted ANOVA are those estimable from the observed (non-missing) plots, so F-tests stay valid when plots are missing. For a complete, connected design they equal the design formulas reported in `anova_df`, which use total plots (`n`), locations (`l`), incomplete blocks (`b`), and genotypes (`v`). Terms with a single level in the data (for example the location term in a single-location trial) are left out of the model and shown with 0 df.

Usage

```
analyze_augmented_prep_design(
  data,
  response,
  genotype_col = "all_entries",
  location_col = "location",
  block_col = "block",
  entry_type_col = "entry_type",
  extra_repeat_col = "is_extra_repeat",
```

```
  repeated_at_col = "repeated_at",
  control_col = "is_control",
  replicate_col = "replicate",
  genotype_by_location = FALSE
)
```

Arguments

`data`	A complete augmented p-rep design data frame, usually `design$design` from `augmented_prep_design()`, or an object returned by `augmented_prep_design()`, with a response column added.
`response`	Name of the numeric response column.
`genotype_col`	Name of the genotype/treatment column.
`location_col`	Name of the location column.
`block_col`	Name of the incomplete block column. Block numbers may repeat across locations; blocks are treated as nested within locations.
`entry_type_col`	Optional entry type column. Used only for genotype summaries.
`extra_repeat_col`	Optional extra-repeat indicator column. Used only for genotype summaries.
`repeated_at_col`	Optional column identifying where each entry is repeated. Used only for genotype summaries.
`control_col`	Optional control flag column. Used only for genotype summaries.
`replicate_col`	Optional replicate (superblock) column. When present and a location has more than one replicate, replicates nested within locations are fitted, and blocks are nested within replicates. Set to `NULL` to ignore it.
`genotype_by_location`	Genotype-by-location interaction in a multi-location analysis: `"none"` (default, or `FALSE`), `"fixed"` (or `TRUE`), or `"random"` (needs the 'lme4' package). With `"fixed"`, adjusted means are averaged over locations. With `"random"`, adjusted means are the genotype BLUEs from the mixed model, `variance_components` gives the REML estimates, and the ANOVA table is the fixed-interaction table, whose genotype-by-location and error mean squares show the size of the interaction.

Value

An object of class `augmented_prep_analysis`: a list with

input, formula, model The column mapping, model formula, and the `lm` fit.

anova, anova_df, anova_formatted Sequential ANOVA (estimable df), design df formulas, and a formatted ANOVA table.

plot_results The input data with fitted values and residuals.

genotype_summary One row per genotype with raw, fitted, and adjusted means and the standard error of the adjusted mean.

adjusted_means Genotypes ranked by adjusted mean (best first).

adjusted_means_vcov Covariance matrix of the estimable adjusted means, used by analyze_augmented_prep_two_stage for weighting.

diagnostics Error df, residual standard deviation, R-squared, mean_sed (mean standard error of a difference between two adjusted means), lsd_5pct (the matching least significant difference), and flags for rank deficiency or a saturated fit.

variance_components, mixed_model For genotype_by_location = "random": REML variance components and the lmerMod fit; otherwise NULL.

message A note when F-tests or adjusted means are not estimable.

Adjusted means

Genotype effects are fixed, so the adjusted means are best linear unbiased estimates (BLUEs): the genotype effect plus the average location and block effect over all observed plots, i.e. the expected value of the genotype in an average plot of the trial. They remove the advantage or penalty of the blocks and locations where each entry happened to be planted, so use adjusted_mean (not raw_mean) to rank and select entries. Differences between adjusted means are the same as those from emmeans on the fitted model. With genotype_by_location = "fixed" the genotype-by-location effects are averaged over locations, weighting each location by its share of the observed plots. With "random", standard errors come from the REML covariance of the fixed effects, which includes the genotype-by-location variance; lsd_5pct uses the error df of the fixed-interaction model as an approximation.

Per-location analysis

Williams, Piepho and Whitaker (2011) suggest analysing each location with an incomplete-block model. Pass the rows of one location, e.g. analyze_augmented_prep_design(data[data$location == "A",], "yield"). augmented_prep_design() keeps the blocks of every location connected, so this is estimable whenever the location has enough p-rep plots.

Examples

```
design <- augmented_prep_design(
  treatments = paste0("G", seq_len(8)),
  locations = c("Loc1", "Loc2"),
  block_size = 4,
  n_duplicate_per_location = c(Loc1 = 2, Loc2 = 2),
  n_candidates = 3,
  seed = 123
)

design_data <- design$design
design_data$yield <- rnorm(nrow(design_data), mean = 100, sd = 10)
analysis <- analyze_augmented_prep_design(design_data, response = "yield")
analysis
head(analysis$adjusted_means)
```

`analyze_augmented_prep_two_stage` *Two-stage analysis of a multi-location augmented p-rep trial*

Description

Analyses each location separately (stage 1) and combines the per-location adjusted means across locations (stage 2), weighting them by their precision. This is the approach suggested by Williams, Piepho and Whitaker (2011, Section 3) for adding spatial error components and for heterogeneous error variances across locations: every location gets its own error variance (and, optionally, its own spatial trend), and the stage-2 weights carry that information into the combined means (Möhring and Piepho, 2009).

Usage

```
analyze_augmented_prep_two_stage(
  data,
  response,
  stage1 = c("blocks", "spatial"),
  weighting = c("full", "diagonal", "none"),
  genotype_by_location = c("random", "none"),
  genotype_col = "all_entries",
  location_col = "location",
  block_col = "block",
  replicate_col = "replicate",
  row_col = "row",
  col_col = "col",
  spatial_nseg = NULL,
  spatial_random_blocks = TRUE
)
```

Arguments

`data`	A design data frame (usually `design$design` from `augmented_prep_design()`), or a design object, with a response column.
`response`	Name of the numeric response column.
`stage1`	Stage-1 model: `"blocks"` or `"spatial"`.
`weighting`	Stage-2 weighting: `"full"`, `"diagonal"`, or `"none"`.
`genotype_by_location`	`"random"` (default) to estimate a genotype-by-location variance in stage 2, or `"none"` to omit it.
`genotype_col`, `location_col`, `block_col`, `replicate_col`	Column names, as in `analyze_augmented_prep_design()`.
`row_col`, `col_col`	Field row and column coordinate columns, used by `stage1 = "spatial"`.

`spatial_nseg` Optional number of P-spline segments for columns and rows, `c(col, row)`. Defaults to half the number of distinct columns and rows of each location (at least 3, at most 40), which is much faster than one segment per row or column and usually as accurate.

`spatial_random_blocks`
Logical. If `TRUE` (default), incomplete blocks are included as a random effect in the spatial model.

Value

An object of class `augmented_prep_two_stage`, a list with

adjusted_means Combined genotype means ranked best first, with standard errors and the number of locations.

variance_components Stage-2 REML genotype-by-location variance.

locations One row per location: stage-1 model, plots, genotypes, residual variance and df, mean SED and, for spatial fits, the effective dimension of the spatial surface.

heterogeneity Hartley's Fmax ratio of the stage-1 residual variances with a simulated 5% critical value.

stage1_means The stage-1 genotype means and standard errors per location.

stage1_fits The stage-1 fits (analysis objects or `SpATS` objects), named by location.

diagnostics Settings, number of stage-2 means, REML log-likelihood, mean SED and approximate 5% LSD.

Stage 1

`stage1 = "blocks"` The incomplete-block model of `analyze_augmented_prep_design()` at each location (replicates and blocks fixed, genotypes fixed).

`stage1 = "spatial"` A spatial model fitted with the 'SpATS' package (Rodríguez-Álvarez et al., 2018): genotypes fixed, a smooth two-dimensional P-spline surface over the field rows and columns (`PSANOVA`), replicates fixed and incomplete blocks random. It needs the `row` and `col` field coordinates (use `field_cols` in `augmented_prep_design()`).

The stage-1 genotype means are BLUEs with their full covariance matrix. Each location needs some error degrees of freedom, i.e. enough p-rep entries for its blocks. In spatial fits part of the plot-to-plot noise is taken up by the smooth surface, so the reported residual variance is somewhat smaller than in the block model; the stage-1 covariance matrix accounts for this, and the Hartley test on spatial residual variances is approximate.

Stage 2

The stage-1 means $\bar{y}_{gl}$ are modelled as $\bar{y}_{gl} = \mu + \lambda_l + \gamma_g + (\gamma\lambda)_{gl} + e_{gl}$, with fixed locations and genotypes, random genotype-by-location effects with variance σ^2_{GL} (estimated by REML) and stage-1 errors e whose covariance is taken from stage 1:

`weighting = "full"` the full stage-1 covariance matrix of each location (most accurate);

`weighting = "diagonal"` only the stage-1 variances (weights $1/se^2$);

`weighting = "none"` an unweighted analysis of the stage-1 means with a common error, which ignores heterogeneity (for comparison).

Genotype adjusted means average the location effects with equal weights.

References

Williams, E., Piepho, H.-P. and Whitaker, D. (2011). Augmented p-rep designs. Biometrical Journal 53, 19-27. doi:10.1002/bimj.201000102

Möhring, J. and Piepho, H.-P. (2009). Comparison of weighting in two-stage analysis of plant breeding trials. Crop Science 49, 1977-1988. doi:10.2135/cropsci2009.02.0083

Rodríguez-Álvarez, M. X., Boer, M. P., van Eeuwijk, F. A. and Eilers, P. H. C. (2018). Correcting for spatial heterogeneity in plant breeding experiments with P-splines. Spatial Statistics 23, 52-71. doi:10.1016/j.spasta.2017.10.003

See Also

`analyze_augmented_prep_design()` for the one-stage analysis.

Examples

```
design <- augmented_prep_design(
  treatments = sprintf("G%02d", 1:60),
  locations = c("A", "B", "C"),
  method = "alpha",
  block_size = 6,
  field_cols = 9,
  seed = 1
)
trial <- design$design
set.seed(2)
# Location C is twice as noisy as A and B.
trial$yield <- 50 + stats::rnorm(nrow(trial), sd = ifelse(trial$location == "C", 4, 2))

fit <- analyze_augmented_prep_two_stage(trial, response = "yield")
fit
fit$locations


if (requireNamespace("SpATS", quietly = TRUE)) {
  spatial <- analyze_augmented_prep_two_stage(trial, "yield", stage1 = "spatial")
  spatial$locations
}
```

analyze_augmented_row_column_design

Analyze an augmented row-column design

Description

Fits the fixed-effects model: `y = mean + rowgroup + colgroup + rowgroup:colgroup + rowgroup:row + colgroup:col + genotype + error`. Sums of squares are sequential, in the order above. Degrees of freedom are the ranks actually estimable from the observed (non-missing) plots. For a complete design they equal the augmented row-column formulas, reported in `anova_df`, which use field rows (`k`), field columns (`s`), row groups (`gk`), column groups (`gs`), check cultivars (`vc`), and unreplicated entries (`ve`): row groups `gk - 1`, rows within row groups `k - gk`, column groups `gs - 1`, columns within column groups `s - gs`, blocks `(gk - 1)(gs - 1)`, genotypes `vc + ve - 1`, and error as the remainder of `k * s - 1`.

Usage

```
analyze_augmented_row_column_design(
  data,
  response,
  genotype_col = "all_entries",
  rowgroup_col = "rowgroup",
  colgroup_col = "colgroup",
  row_col = "row",
  col_col = "col",
  type_col = "type",
  control_col = "is_control",
  random_blocks = FALSE,
  entries = c("fixed", "random"),
  spatial = FALSE,
  spatial_nseg = NULL
)
```

Arguments

`data`	A complete augmented row-column design data frame, usually `design$design` from `augmented_row_column_design()`, with a response column added.
`response`	Name of the numeric response column.
`genotype_col`	Name of the genotype/treatment column.
`rowgroup_col`	Name of the row-group column.
`colgroup_col`	Name of the column-group column.
`row_col`	Name of the field row column.
`col_col`	Name of the field column column.
`type_col`	Optional entry type column. Used only for genotype summaries.

control_col	Optional control flag column. Identifies the checks for entries = "random"; otherwise used only for genotype summaries.
random_blocks	Logical. If TRUE, rows within row groups, columns within column groups and blocks are random effects, which recovers inter-block information (Wolfinger et al., 1997; Möhring et al., 2015). Row and column groups stay fixed. Fitted by REML with 'lme4'.
entries	"fixed" (default) or "random". With "random", checks are fixed and the unreplicated entries are random effects from a common breeding population (Piepho, Williams and Fleck, 2006); entry values are then BLUPs. Fitted by REML with 'lme4'.
spatial	Logical. If TRUE, the plot errors get a spatial add-on: a two-dimensional P-spline surface over rows and columns fitted with the 'SpATS' package, with row and column groups fixed and rows, columns and blocks random. Genotypes are fixed.
spatial_nseg	Optional number of P-spline segments for columns and rows, c(col, row). Defaults to half the number of columns and rows (at least 3, at most 40).

Details

Unreplicated entries each fit their own plot exactly, so the model is fitted without a dense genotype design matrix: the residual comes from the replicated (check) plots alone. model is the corresponding lm fit on the check plots, with aliased columns removed. This keeps the analysis fast for large trials.

Value

An object of class augmented_row_column_analysis: a list with

input, formula, model The column mapping, model formula, and an lm fit on the check plots.

anova, anova_df, anova_formatted Sequential ANOVA (estimable df), design df formulas, and a formatted ANOVA table.

plot_results The input data with fitted values and residuals.

genotype_summary One row per genotype with raw, fitted, and adjusted means and the standard error of the adjusted mean.

adjusted_means Genotypes ranked by adjusted mean (best first).

diagnostics Error df, residual standard deviation, R-squared, mean_sed (mean standard error of a difference between two adjusted means), lsd_5pct (the matching least significant difference), and flags for rank deficiency or a saturated fit.

variance_components, mixed_model For mixed or spatial fits: the REML variance components and the lmerMod or SpATS fit; otherwise NULL.

message A note when F-tests or adjusted means are not estimable.

Mixed and spatial models

The ANOVA table always comes from the fixed-effects model of Piepho and Williams (2016, Eq. 1). With random_blocks, entries = "random" or spatial, the adjusted means, their standard

errors, `mean_sed` and `lsd_5pct` come from the corresponding REML fit instead. Standard errors are exact prediction standard errors from the mixed-model equations for the estimated variance components; `lsd_5pct` uses the fixed-model error df as an approximation.

Adjusted means

With the default fixed-effects model, genotype effects are fixed, so the adjusted means are best linear unbiased estimates (BLUEs). Each is the genotype effect plus the average row, column, and block effect over all observed plots, i.e. the expected value of the genotype in an average plot of the field. For an unreplicated entry this is its observed value corrected for the row, column, and block effects of its plot, estimated from the checks; its standard error combines the plot error and the uncertainty of that correction. Use `adjusted_mean` (not `raw_mean`) to rank and select entries.

Examples

```
design <- augmented_row_column_design(
  treatments = paste0("G", seq_len(108)),
  controls = paste0("Check", seq_len(4)),
  rows_in_field = 12,
  cols_in_field = 12,
  rows_per_block = 4,
  cols_per_block = 4,
  n_candidates = 3,
  seed = 123
)

design_data <- design$design
design_data$yield <- rnorm(nrow(design_data), mean = 100, sd = 10)
analysis <- analyze_augmented_row_column_design(design_data, response = "yield")
analysis
head(analysis$adjusted_means)
```

`analyze_environment_anova` *Run individual ANOVA by environment*

Description

Run individual ANOVA by environment

Usage

```
analyze_environment_anova(pheno)
```

Arguments

pheno	Standardized phenotype data from `prepare_breedbase_pheno()`.

Value

A tibble with one ANOVA summary row per environment.

Examples

```
df <- data.frame(
  environment = factor(rep(c("Loc1", "Loc2"), each = 8)),
  genotype = factor(rep(rep(c("G1", "G2"), each = 4), times = 2)),
  replicate = factor(rep(rep(c(1, 2), each = 2), times = 4)),
  block = factor(1),
  value = c(10, 11, 12, 13, 14, 15, 16, 17,
            15, 16, 17, 18, 19, 20, 21, 22)
)

analyze_environment_anova(df)
```

---

analyze_one_environment

Analyze one environment using ANOVA

Description

Runs an individual ANOVA for one environment. If more than one replicate is present, the model is `value ~ replicate + genotype`; otherwise, the model is `value ~ genotype`.

Usage

```
analyze_one_environment(df)
```

Arguments

`df`	A data frame containing one environment with columns `environment`, `genotype`, `replicate`, and `value`.

Value

A tibble with ANOVA summary statistics.

Examples

```
df <- data.frame(
  environment = "Loc1",
  genotype = factor(rep(c("G1", "G2", "G3"), times = 2)),
  replicate = factor(rep(1:2, each = 3)),
  value = c(10, 12, 15, 11, 13, 14)
)

analyze_one_environment(df)
```

`analyze_resolvable_row_column` *Analyze a resolvable row-column trial with RC, LV + RC or AR + RC models*

Description

Fits the variance models of Williams, John and Whitaker (2006) by residual maximum likelihood (REML), with replicates and treatments fixed:

$$E(y) = R\pi + X\tau, \quad var(y) = \sigma^2 V.$$

`model = "rc"` The standard resolvable row-column mixed model: $V = I + \eta_R P_R + \eta_C P_C$, i.e. random rows and columns within replicates.

`model = "lv"` The two-dimensional linear variance model (LV + RC, Eq. 2): $V = I + \eta_R P_R + \eta_C P_C - (\phi_R F_R + \phi_C F_C + \phi_{RC} F_{RC})$, with LV matrices for plots in the same row, in the same column, and in neither. `lv_terms` chooses which ϕ terms are fitted and `equal_phi = TRUE` uses one ϕ for rows and columns (square plots).

`model = "ar"` Rows and columns plus a separable first-order autoregressive process (AR + RC, Eq. 4): $V = I + \eta_R P_R + \eta_C P_C + \psi D(\rho_C) \otimes D(\rho_R)$ within replicates.

In every model the identity term is the independent error ("nugget"). With `compare = TRUE` the nested sequence RC, LV with one ϕ, LV with ϕ_R, ϕ_C, LV with ϕ_{RC} as well, and AR + RC are all fitted and compared by REML deviance (likelihood-ratio tests for the nested LV terms, and AIC), which is the sequential fitting process the paper recommends for deciding whether spatial terms are needed.

Usage

```
analyze_resolvable_row_column(
  data,
  response,
  model = c("rc", "lv", "ar"),
  lv_terms = c("phi_r", "phi_c"),
  equal_phi = FALSE,
  long_col = NULL,
  compare = TRUE,
  genotype_col = "all_entries",
  replicate_col = "replicate",
  row_col = "row_in_rep",
  col_col = "col_in_rep"
)
```

Arguments

`data`	A design data frame (for example `design$design` from `resolvable_row_column_design()`) or a design object, with a response.

`response`	Name of the numeric response column.
`model`	Variance model reported in `adjusted_means`: `"rc"`, `"lv"`, or `"ar"`.
`lv_terms`	For `model = "lv"`, the spatial terms to fit: any of `"phi_r"`, `"phi_c"`, `"phi_rc"`.
`equal_phi`	Logical. For `model = "lv"`, use a single ϕ for rows and columns.
`long_col`	Optional name of a column with long-column (or long-row) labels to fit as fixed effects, as in latinized designs.
`compare`	Logical. If `TRUE`, also fit and compare the sequence of models.
`genotype_col, replicate_col, row_col, col_col`	Column names.

Details

For latinized designs the long columns (or long rows) can be fitted as fixed effects with `long_col`.

Value

An object of class `resolvable_row_column_analysis`, a list with

plot_results One row per plot: the data, the fitted value (fixed effects plus the predicted row, column and spatial effects) and the residual $V^{-1}(y - X\hat{\beta})$.

adjusted_means Treatment BLUEs (average over replicates) with standard errors, ranked best first.

variance_parameters The REML estimates: σ^2, the η, ϕ, ψ and ρ parameters, and the implied row and column variance components $\sigma^2\eta_R/s$ and $\sigma^2\eta_C/k$.

model_comparison With `compare = TRUE`, the deviance table.

anova Intra-block ANOVA from the fixed model `replicate + rows(replicate) + columns(replicate) + treatment`.

diagnostics REML deviance, number of variance parameters, mean SED and approximate 5% LSD.

References

Williams, E. R., John, J. A. and Whitaker, D. (2006). Construction of resolvable spatial row-column designs. Biometrics 62, 103-108. doi:10.1111/j.15410420.2005.00393.x

Examples

```
design <- resolvable_row_column_design(paste0("T", 1:20), 3, 4, 5,
                                       n_starts = 1, n_iter = 300, seed = 1)
trial <- design$design
set.seed(2)
trial$y <- 10 + 0.3 * trial$row + stats::rnorm(nrow(trial))
fit <- analyze_resolvable_row_column(trial, "y", model = "lv")
fit$model_comparison
head(fit$adjusted_means)
```

`analyze_single_stage` *Single-stage analysis of several trials*

Description

Fits all trials (locations, environments) together in one mixed model and gives the genotype adjusted means across trials, to compare with (or use instead of) the two-stage analysis of `analyze_two_stage()`:

$$y = \mu + \text{trial} + \text{genotype} + \text{rep(trial)} + \text{block(rep, trial)} + \text{genotype} \times \text{trial} + e,$$

with trials and genotypes fixed, and replicates, blocks and the genotype-by-trial interaction random (fitted with 'lme4'). Genotype means average the trial effects with equal weights. Without 'lme4' the model is fitted with all effects fixed and no genotype-by-trial term.

Usage

```
analyze_single_stage(
  data,
  response,
  genotype_col = "accession",
  location_col = "trial_name",
  replicate_col = NULL,
  block_col = NULL,
  genotype_by_location = c("random", "none"),
  spatial = FALSE,
  row_col = "row",
  col_col = "col",
  spatial_nseg = NULL
)
```

Arguments

`data`	A data frame with one row per plot, e.g. from `breedbase_phenotypes()`.
`response`	Name of the numeric response column.
`genotype_col`, `location_col`	Genotype and trial (location) columns.
`replicate_col`, `block_col`	Optional replicate and block columns; by default `"rep"`/`"replicate"` and `"block"` when the data have them.
`genotype_by_location`	`"random"` (default) or `"none"`.
`spatial`	Logical. Fit the single-stage model with 'SpATS' (needs the field `row` and `col` of every plot).
`row_col`, `col_col`	Field row and column columns (for `spatial = TRUE`).
`spatial_nseg`	Optional number of P-spline segments, `c(col, row)`.

Details

With `spatial = TRUE` the model is fitted with 'SpATS': genotypes and trials fixed, replicates within trials fixed, blocks and the genotype-by-trial interaction random, and a smooth P-spline surface over the field rows and columns. SpATS fits one surface per model, so the trials are placed side by side in one coordinate system, separated by empty columns: each trial keeps its own spatial trend and the trends do not run from one trial into another.

One error variance is shared by all trials here; when trials differ much in precision, the weighted two-stage analysis is the better choice (Möhring and Piepho, 2009).

Value

An object of class `single_stage_analysis`: `adjusted_means` (ranked, with standard errors and the number of trials), `adjusted_means_vcov`, `variance_components`, `plot_results` (fitted values and residuals), `diagnostics` and the fitted `model`.

References

Möhring, J. and Piepho, H.-P. (2009). Comparison of weighting in two-stage analysis of plant breeding trials. Crop Science 49, 1977-1988. doi:10.2135/cropsci2009.02.0083

Examples

```
sites <- c("North", "South")
met <- do.call(rbind, lapply(1:2, function(i) {
  d <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4), 12, 12, 4, 4,
                                   n_candidates = 5, seed = i)$design
  d$location <- sites[i]
  d
}))
set.seed(1)
g <- stats::setNames(stats::rnorm(112, sd = 3), c(paste0("G", 1:108), paste0("C", 1:4)))
met$yield <- 50 + g[met$all_entries] + stats::rnorm(nrow(met))
one <- analyze_single_stage(met, "yield", genotype_col = "all_entries", location_col = "location")
one
```

---

analyze_two_stage *Two-stage analysis of a multi-location trial, for any design*

Description

Analyses each location on its own (stage 1) and combines the genotype means across locations (stage 2), weighting them by their precision (Möhring and Piepho, 2009). Works for every design of the package grown at several locations: augmented row-column, augmented p-rep and resolvable row-column designs. For augmented p-rep designs, `analyze_augmented_prep_two_stage()` gives the same analysis with the p-rep block model of the paper.

Usage

```
analyze_two_stage(
  data,
  response,
  stage1 = c("spatial", "blocks", "design"),
  weighting = c("full", "diagonal", "none"),
  genotype_by_location = c("random", "none"),
  genotype_col = "all_entries",
  location_col = "location",
  row_col = "row",
  col_col = "col",
  replicate_col = NULL,
  block_col = NULL,
  spatial_nseg = NULL
)
```

Arguments

`data`	A data frame with all locations (one row per plot), with the design columns and a response column.
`response`	Name of the numeric response column.
`stage1`	`"spatial"`, `"blocks"` or `"design"`.
`weighting`	`"full"`, `"diagonal"` or `"none"`.
`genotype_by_location`	`"random"` or `"none"`.
`genotype_col`, `location_col`	Genotype and location columns.
`row_col`, `col_col`	Field row and column coordinates (spatial stage 1).
`replicate_col`, `block_col`	Optional replicate (fixed) and block (random) columns of the spatial stage 1. By default `"replicate"` and `"block"` when the data have them.
`spatial_nseg`	Optional number of P-spline segments, `c(col, row)`.

Value

An object of class `two_stage_analysis`: a list with `adjusted_means` (combined, ranked), `variance_components`, `locations` (one row per location), `heterogeneity` (Hartley's Fmax), `stage1_means`, `stage1_fits` (per location, with `plot_results`, for `first_stage_values()` and `compare_first_stage()`) and `diagnostics`.

Stage 1

`stage1 = "spatial"` **(default)** A 'SpATS' spatial model at every location (Rodríguez-Álvarez et al., 2018): genotypes fixed, a smooth two-dimensional P-spline surface over the field rows and columns, replicates fixed and blocks random when `replicate_col` and `block_col` are given.

`stage1 = "blocks"` A linear model at every location that needs no design object: replicates and blocks within replicates fixed (when `replicate_col` and `block_col` are given), genotypes fixed. For data read from Breedbase without field coordinates.

`stage1 = "design"` The design's own model at every location: `analyze_augmented_row_column_design()` for augmented row-column designs, `analyze_resolvable_row_column()` (row-column model) for resolvable row-column designs, and the block model of `analyze_augmented_prep_design()` for p-rep designs. The design is recognised from its columns.

The stage-1 genotype means are BLUEs with their full covariance matrix.

Stage 2

As in `analyze_augmented_prep_two_stage()`: fixed locations and genotypes, random genotype-by-location effects (REML), and stage-1 errors with the stage-1 covariance (`weighting = "full"`), only the stage-1 variances (`"diagonal"`) or a common error (`"none"`, for comparison).

References

Möhring, J. and Piepho, H.-P. (2009). Comparison of weighting in two-stage analysis of plant breeding trials. Crop Science 49, 1977-1988. doi:10.2135/cropsci2009.02.0083

Rodríguez-Álvarez, M. X., Boer, M. P., van Eeuwijk, F. A. and Eilers, P. H. C. (2018). Correcting for spatial heterogeneity in plant breeding experiments with P-splines. Spatial Statistics 23, 52-71. doi:10.1016/j.spasta.2017.10.003

Examples

```
# The same augmented row-column layout, randomized at three locations.
sites <- c("North", "Central", "South")
trials <- lapply(seq_along(sites), function(i) {
  d <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                   12, 12, 4, 4, n_candidates = 5, seed = i)$design
  d$location <- sites[i]
  d
})
met <- do.call(rbind, trials)
set.seed(1)
g <- stats::setNames(stats::rnorm(112, sd = 3), c(paste0("G", 1:108), paste0("C", 1:4)))
met$yield <- 50 + g[met$all_entries] + 0.5 * met$row + stats::rnorm(nrow(met))
fit <- analyze_two_stage(met, "yield", stage1 = "design")
fit

if (requireNamespace("SpATS", quietly = TRUE)) {
  spatial <- analyze_two_stage(met, "yield")      # SpATS at every location
  spatial$locations
}
```

`as_breedbase_design` *Convert an agricolae field book for saving to Breedbase*

Description

Adapts a design generated by the CRAN package **agricolae** to `save_breedbase()` and `phenotype_template()`. The randomization is kept; no design is generated or randomized by this function. Conversion is entirely offline and does not require agricolae to be installed. To also record the trial name, location, breeding program and year for a later save, use `prepare_breedbase_trial()`.

Usage

```
as_breedbase_design(
  design,
  accession = NULL,
  checks = NULL,
  design_type = NULL,
  plot = NULL,
  block = NULL,
  rep = NULL,
  row = NULL,
  col = NULL
)
```

Arguments

`design`	An agricolae design (a list with `book` and `parameters`), or its field-book data frame, including the result of `agricolae::zigzag()`.
`accession`	Name of the field-book column containing Breedbase accession names. Inferred only when exactly one non-structural column is present. For factorial, split-plot, strip-plot and Graeco-Latin designs, explicitly select the genotype column; combinations of treatment factors are not turned into accession names.
`checks`	Character vector of check accession names. By default, uses `parameters$trt1` for an agricolae augmented block design (`design.dau()`), and no checks for other designs. Use `character(0)` to explicitly mark no checks.
`design_type`	Breedbase design type. Inferred for `crd` (`"CRD"`), `rcbd` (`"RCBD"`), `alpha` (`"Alpha"`), `lattice` (`"Lattice"`), `dau` (`"Augmented"`), `split` (`"splitplot"`) and `strip` (`"stripplot"`); factorial designs use their underlying CRD or RCBD. Required for a bare field book or a design without a direct Breedbase type, such as Latin squares, Youden, BIB and cyclic designs. Select a compatible type accepted by your Breedbase installation; the original agricolae type remains in the returned object's `parameters`.
`plot`, `block`, `rep`, `row`, `col`	Optional field-book column names. Defaults detect `plots`, `block`, a replicate column (`replication`, `replicate`, `rep` or `r`), and the pair `row`, `col`. Missing block or replicate columns are set to 1; in RCBD, block also supplies replicate.

Augmented blocks are not treated as replicates. Coordinates are used only when both columns are available, and are never inferred from plot numbers or the sketch. Mapped numeric columns must contain positive integers.

Details

Plot numbers, accessions, blocks, replicates and any explicit coordinates retain their field-book values. Alpha designs' `cols` column describes positions within blocks and is not assumed to be a field coordinate. For split plots, where `plots` is repeated across subplots, sequential unique plot numbers are assigned in field-book row order. An explicit `plot` mapping must always identify each row uniquely.

The original field book and generation parameters are retained as `book` and `parameters`. Saving uploads one plot per row, with accession, block, replicate, check status and optional coordinates. Other treatment factors and the main-plot/subplot hierarchy are retained locally, but are not uploaded by `save_breedbase()`. In particular, this conversion does not create treatment records or an analysis model for factorial designs.

Accession names must already exist in Breedbase. Supply the new trial's name, year, location, description and breeding program to `save_breedbase()`; use `dry_run = TRUE` to inspect or check before saving.

Value

A `breedbase_design` list with the standardized plot table (`design`), Breedbase `design_type`, original book and `parameters`. Pass the whole object to `save_breedbase()` or `phenotype_template()`.

See Also

`save_breedbase()`, `phenotype_template()`

Examples

```
if (requireNamespace("agricolae", quietly = TRUE)) {
  accessions <- c("G1", "G2", "G3", "CHECK_A")
  randomized <- agricolae::design.rcbd(accessions, r = 3, seed = 2026)
  converted <- as_breedbase_design(randomized, checks = "CHECK_A")
  payload <- save_breedbase(converted, trial_name = "RCBD_2026",
    year = 2026, location = "Ibadan", description = "RCBD from agricolae",
    dry_run = TRUE)
 sheet <- phenotype_template(converted, "RCBD_2026", traits = "yield", location = "Ibadan")
}
```

`augmented_prep_concurrence` *Calculate pairwise concurrence for a p-rep design*

Description

Counts how many times each pair of entries appears together in the same incomplete block across all locations.

Usage

```
augmented_prep_concurrence(design, treatments = NULL)
```

Arguments

design	A p-rep design data frame from augmented_prep_design().
treatments	Optional treatment names defining matrix order.

Value

A symmetric integer matrix of pairwise block concurrences.

Examples

```
design <- augmented_prep_design(
  treatments = paste0("G", seq_len(12)),
  locations = c("Loc1", "Loc2"),
  block_size = 4,
  n_candidates = 5,
  seed = 1
)
conc <- augmented_prep_concurrence(design$design)
table(conc[upper.tri(conc)])
```

---

augmented_prep_design *Generate an augmented p-rep design*

Description

Creates a multi-location augmented partially replicated (p-rep) design (Williams, Piepho and Whitaker, 2011). Instead of replicated checks, a subset of the test entries is replicated at each location: every entry is planted once at every location, and the entries chosen for p-rep at a location get extra plots there. All plots of a location are arranged in incomplete blocks, chosen so that pairs of entries share a block at most once across the whole trial ((0,1)-designs).

Usage

```
augmented_prep_design(
  treatments,
  locations,
  block_size = 8,
  prep_prop = NULL,
  n_duplicate_per_location = NULL,
  replications_per_repeated_entry = 2,
  n_blocks_per_location = NULL,
  resolvable = FALSE,
  n_candidates = 1000,
```

```
  seed = NULL,
  field_cols = NULL,
  rows_per_block = NULL,
  blocks_per_row = NULL,
  serpentine = TRUE,
  output_file = NULL,
  method = c("search", "alpha", "alpha2"),
  n_prep_per_block = NULL,
  n_efficiency_candidates = 5,
  trial_name = NULL,
  plot_name_pattern = "{trial}_{location}_{accession}_{block}_{rep}"
)
```

Arguments

`treatments`	Character vector of entry names.
`locations`	Character vector of location names.
`block_size`	Maximum final incomplete block size. Can be a single integer or a named vector indexed by location (the algebraic methods use its smallest value). Actual block sizes may differ by one plot when totals do not divide evenly.
`prep_prop`	Optional proportion of entries to duplicate per location (`method = "search"` only). If `NULL`, entries are split across locations so each entry is duplicated at approximately one location.
`n_duplicate_per_location`	Optional number of entries duplicated at each location (`method = "search"` only). Can be a single integer, a vector matching `locations`, or a named vector indexed by location.
`replications_per_repeated_entry`	Total number of plots for entries selected for p-rep replication at a location. The default `2` gives one base plot plus one extra plot. Can be a single integer or a named vector indexed by location. Must be 2 for the algebraic methods.
`n_blocks_per_location`	Optional number of incomplete blocks at each location (`method = "search"` only). If `NULL`, the number of blocks is the minimum needed so no block is larger than `block_size`. Can be a single integer or a named vector indexed by location.
`resolvable`	Logical (`method = "search"`). If `TRUE`, each location is laid out as a resolvable design: the blocks are grouped into `replications_per_repeated_entry` replicates, each replicate holds every p-rep entry of that location exactly once plus an even share of the unreplicated entries, and all replicates have the same number of blocks. If `n_blocks_per_location` is given it must be a multiple of the number of replicates. Designs from the algebraic methods are always resolvable.
`n_candidates`	Maximum number of candidate designs (`"search"`) or of random starting arrays (`"alpha"`, `"alpha2"`) to generate. `summary$n_candidates_evaluated` reports how many were used.

`seed` Optional random seed.

`field_cols` Optional number of field columns for row/column layout. Can be a single integer or a named vector indexed by location.

`rows_per_block` Optional number of physical field rows each incomplete block should occupy. Can be a single integer or a named vector for selected locations. Must be used with `blocks_per_row` for those locations.

`blocks_per_row` Optional number of incomplete blocks to place side by side in each field row band. Can be a single integer or a named vector for selected locations. Must be used with `rows_per_block` for those locations.

`serpentine` Logical. If `TRUE`, row/column coordinates use serpentine ordering within each location.

`output_file` Optional tab-delimited output file for the complete design.

`method` Construction method: `"search"`, `"alpha"` or `"alpha2"` (see Details).

`n_prep_per_block`
For the algebraic methods, the number `m` of p-rep entries in every block. If `NULL`, the largest feasible `m` whose blocks fit in `block_size` is used.

`n_efficiency_candidates`
Number of optimal candidates (no entry repeated within a block, connected locations, and no pair of entries sharing more than one block) to compare on the average efficiency factor; the one with the highest efficiency is returned. Use `1` to return the first optimal candidate. Efficiency is compared for up to 2000 entries.

`trial_name` Optional Breedbase trial name. When given, a `plot_name` column with the plot names used by `save_breedbase()` is added to the design (see `add_plot_names()`), so the names can be checked with `breedbase_check()` and changed before saving. For a multi-location design the location is added, one trial per location.

`plot_name_pattern`
Pattern for the plot names, as in `add_plot_names()`. Used only with `trial_name`; `{location}` is the design location.

Details

Three construction methods are available through `method`:

`"search"` **(default)** A flexible randomized search. Any number of entries, locations, block sizes, p-rep proportions (`prep_prop`, `n_duplicate_per_location`), replication levels (`replications_per_repeated_entry`) and block counts can be used, with an optional resolvable layout (`resolvable`).

`"alpha"` The partial alpha-array construction of Section 2.1 of the paper. An alpha-array for `l * m` rows and `2l` replicates is searched, the `m x 2` rectangles on its diagonal give the two-replicate design for the p-rep entries of each location, and half of the remaining elements of each column are deleted (Section 2.2) so that the design is a (0,1)-design. Every entry is duplicated at exactly one location, each location is resolvable with two replicates of `s` blocks, and each block holds `m` p-rep entries; the final block size is `(l + 1) m / 2` (blocks of the two replicates differ by one plot when this is not an integer).

`"alpha2"` The augmented alpha(2)-array construction of Section 2.4, for even `m` and an odd number of blocks per replicate `s`. Every block has `(l + 1) m / 2` plots.

For the algebraic methods, `m` is set with `n_prep_per_block` (or chosen as the largest value whose blocks fit in `block_size`), the number of blocks and of duplicated entries follow from the construction, and `replications_per_repeated_entry` must be 2. When the number of entries is not a multiple of the array size, the surplus array positions are dropped as dummy entries, so some blocks are one plot smaller. `augmented_prep_parameters()` lists the feasible parameter sets.

The incomplete block structure is independent from the field geometry. By default, `field_cols` lays the ordered plots into rows and columns. For a rectangular block layout, use `rows_per_block` and `blocks_per_row` to place multiple blocks side by side within each location.

With `method = "search"`, the p-rep plots of every location are spread evenly over its blocks, and the p-rep entries are placed so that they link all blocks of the location (the location is connected), so entry effects can also be estimated from each location on its own. A warning is given when there are too few p-rep plots to connect the blocks. After the random greedy search, the best candidates are improved by swapping plots between blocks to remove pairs of entries that share more than one block.

For all methods, `summary` reports the maximum pairwise concurrence, the number of locations whose blocks are not connected, and the average efficiency factor `efficiency_factor` of the block design (Section 2.3 of the paper; higher is better), which can be used to compare designs. Entry labels, block order within replicates and plot order within blocks are randomized.

Value

A list with design inputs, summary diagnostics, complete plot-level design, Breedbase-compatible design columns, block summary, replication summary, repeated-entry groups, and pairwise concurrence matrix. In the complete design, `repeated_at` is the location where the entry receives its extra p-rep plot(s), or `"not_repeated"` when the entry is not selected for extra replication. `replicate` is the replicate (superblock) of the plot for resolvable designs, and 1 otherwise; the Breedbase `rep` column is `replicate` for resolvable designs and the plot's occurrence number otherwise.

References

Williams, E., Piepho, H.-P. and Whitaker, D. (2011). Augmented p-rep designs. Biometrical Journal 53, 19-27. doi:10.1002/bimj.201000102

See Also

`augmented_prep_parameters()`, `analyze_augmented_prep_design()`, `plot_augmented_prep_design()`

Examples

```
design <- augmented_prep_design(
  treatments = paste0("G", seq_len(12)),
  locations = c("Loc1", "Loc2", "Loc3"),
  block_size = c(Loc1 = 5, Loc2 = 5, Loc3 = 6),
  n_blocks_per_location = c(Loc1 = 4, Loc2 = 4, Loc3 = 4),
  n_duplicate_per_location = c(Loc1 = 4, Loc2 = 2, Loc3 = 4),
  replications_per_repeated_entry = c(Loc1 = 2, Loc2 = 3, Loc3 = 2),
  n_candidates = 5,
  seed = 123,
  field_cols = 8
```

```
)

design$summary
head(design$design)

# Example 1 of Williams, Piepho and Whitaker (2011): 20 entries, two
# locations, blocks of 3 built from a partial alpha-array.
alpha <- augmented_prep_design(
  treatments = paste0("G", 1:20),
  locations = c("L1", "L2"),
  method = "alpha",
  block_size = 3,
  seed = 1
)
alpha$summary[, c("method", "m", "s", "max_pairwise_concurrence", "efficiency_factor")]
```

`augmented_prep_parameters` *Feasible parameters for augmented p-rep constructions*

Description

Lists the parameter sets that `augmented_prep_design()` can build for a given number of entries and locations, in the spirit of Table 3 of Williams, Piepho and Whitaker (2011). For each number `m` of p-rep entries per block, it gives the number of blocks per replicate `s`, the final block size(s), the number of entries duplicated at each location, and how many "dummy" entries are dropped when the number of entries is not a multiple of the array size (dropping them makes some blocks one plot smaller).

Usage

```
augmented_prep_parameters(n_treatments, n_locations, max_block_size = 20)
```

Arguments

`n_treatments`	Number of entries (`v`).
`n_locations`	Number of locations (`l`).
`max_block_size`	Largest final block size to list.

Value

A tibble with one row per method and `m`: `method`, `m`, `s`, `block_size` (largest final block), `min_block_size`, `blocks_per_location`, `plots_per_location` (largest), `duplicated_per_location` (largest), `dummies`, `feasible`, and `note`.

References

Williams, E., Piepho, H.-P. and Whitaker, D. (2011). Augmented p-rep designs. Biometrical Journal 53, 19-27. doi:10.1002/bimj.201000102

Examples

```
augmented_prep_parameters(n_treatments = 120, n_locations = 6, max_block_size = 10)
```

`augmented_row_column_capacity`
Calculate augmented row-column design capacity

Description

Reports the plot counts, error degrees of freedom and feasibility of an augmented row-column layout with superimposed blocks (Piepho and Williams, 2016). Rows are grouped into row groups and columns into column groups; their intersections are the blocks. Groups can have equal sizes (`rows_per_block`, `cols_per_block`) or unequal sizes (`rows_per_rowgroup`, `cols_per_colgroup`), and blocks can hold any number of check plots (`checks_per_block`).

Usage

```
augmented_row_column_capacity(
  rows_in_field = NULL,
  cols_in_field = NULL,
  rows_per_block = NULL,
  cols_per_block = NULL,
  n_controls,
  rows_per_rowgroup = NULL,
  cols_per_colgroup = NULL,
  checks_per_block = NULL
)
```

Arguments

`rows_in_field`	Number of rows in the whole field.
`cols_in_field`	Number of columns in the whole field.
`rows_per_block`	Number of rows in each row group (equal groups).
`cols_per_block`	Number of columns in each column group (equal groups).
`n_controls`	Number of replicated checks/controls (`vc`).
`rows_per_rowgroup`	Optional vector with the number of rows of each row group (`kb(i)`), for unequal groups. Replaces `rows_per_block`.

`cols_per_colgroup`
: Optional vector with the number of columns of each column group (`sb(j)`). Replaces `cols_per_block`.

`checks_per_block`
: Optional number of check plots per block (`pb(i, j)`): a single number, a matrix with one row per row group and one column per column group, or a vector by row group. Defaults to `n_controls` (every block a complete block of checks). Smaller values give incomplete blocks; larger values give extra check replication.

Value

A one-row tibble with the field size, number of groups and blocks, check and entry plots, `error_df`, the error degrees of freedom of the row-column ANOVA, `check_plots - n_controls - (rows - 1) - (cols - 1) - (row` (with fixed rows, columns and blocks a design needs `error_df > 0`; with `error_df = 0`, as in Fig. 2 of Piepho and Williams (2016), analyze with random blocks or random entries; Clarke and Stefanova (2011) recommend about 50), `check_fraction`, and the smallest slack of the row-group and column-group conditions of Piepho and Williams (2016), which must be non-negative for the row-group and column-group sub-designs to be estimable on their own.

References

Piepho, H.-P. and Williams, E. R. (2016). Augmented row-column designs for a small number of checks. Agronomy Journal 108, 2256-2262. doi:10.2134/agronj2016.06.0325

Examples

```
augmented_row_column_capacity(
  rows_in_field = 12,
  cols_in_field = 12,
  rows_per_block = 4,
  cols_per_block = 4,
  n_controls = 4
)

# Example 3 of Piepho and Williams (2016): unequal groups and blocks.
augmented_row_column_capacity(
  n_controls = 4,
  rows_per_rowgroup = c(6, 5, 6, 5),
  cols_per_colgroup = c(3, 4, 3),
  checks_per_block = rbind(c(4, 5, 4), c(3, 5, 3), c(3, 5, 4), c(3, 5, 3))
)
```

`augmented_row_column_design` *Generate an augmented row-column field design*

Description

Randomizes unreplicated treatments and replicated controls (checks) in an augmented row-column design with superimposed blocks (Piepho and Williams, 2016). Adjacent rows form row groups and adjacent columns form column groups; each intersection is a block that receives a given number of check plots. Row, column and block effects are all estimable from the checks, and the layout must leave positive error degrees of freedom (see `augmented_row_column_capacity()`).

Usage

```
augmented_row_column_design(
  treatments,
  controls,
  rows_in_field = NULL,
  cols_in_field = NULL,
  rows_per_block = NULL,
  cols_per_block = NULL,
  plot_type = c("serpentine", "cartesian"),
  n_candidates = 1000,
  seed = NULL,
  output_file = NULL,
  rows_per_rowgroup = NULL,
  cols_per_colgroup = NULL,
  checks_per_block = NULL,
  method = c("auto", "latin", "optimal"),
  trial_name = NULL,
  location = NULL,
  plot_name_pattern = "{trial}_{location}_{accession}_{block}_{rep}"
)
```

Arguments

`treatments`	Character vector of unreplicated treatment/entry names. Its length must equal the number of entry plots.
`controls`	Character vector of replicated check/control names.
`rows_in_field`	Number of rows in the whole field.
`cols_in_field`	Number of columns in the whole field.
`rows_per_block`	Number of rows in each row group (equal groups).
`cols_per_block`	Number of columns in each column group (equal groups).
`plot_type`	Plot numbering order. Use `"serpentine"` or `"cartesian"`.
`n_candidates`	Number of candidate layouts (`"latin"`) or random restarts of the local search (`"optimal"`, at most 50; the search stops after 3 restarts without improvement).
`seed`	Optional random seed.
`output_file`	Optional path for writing the complete design as a tab-delimited file.
`rows_per_rowgroup`, `cols_per_colgroup`	Optional vectors of unequal group sizes, as in `augmented_row_column_capacity()`.

`checks_per_block`
: Optional number of check plots per block, as in `augmented_row_column_capacity()`.

`method`
: Check allocation method: `"auto"`, `"latin"` or `"optimal"`.

`trial_name`
: Optional Breedbase trial name. When given, a `plot_name` column with the plot names used by `save_breedbase()` is added to the design (see `add_plot_names()`), so the names can be checked with `breedbase_check()` and changed before saving.

`location`
: Breedbase location of the trial, for the `{location}` field of the plot names. Used only with `trial_name`.

`plot_name_pattern`
: Pattern for the plot names, as in `add_plot_names()`. Used only with `trial_name`.

Details

Two check-allocation methods are available:

`"latin"` For equal groups where every block is a complete block of checks (one copy of each). Checks are placed with covering Latin rectangles, so controls never share a row or column within a block, the same control never appears twice in a field row or column, and every field row and column has a check. Candidates are scored for spatial spread and row/column balance.

`"optimal"` The general strategy of Piepho and Williams (2016): unequal group sizes and any number of check plots per block (incomplete blocks or extra replication of checks). Check labels are first allocated to blocks balancing row groups, column groups and blocks (Step 2); check plots are then placed within blocks and the layout improved by a local search that maximizes the D-criterion of the check-plot model `row + column + rowgroup:colgroup + check` (Steps 3-5). Check plots next to each other (same column in consecutive rows, or same row in consecutive columns) are avoided, inside blocks above all, and spatial spread breaks ties. Entries fill the remaining plots at random (Step 6).

`method = "auto"` uses `"latin"` when it applies and `"optimal"` otherwise.

Value

A list with:

input Validated design inputs.

summary Design size, error df, method, and the D-criterion `d_criterion` (`det(X'X)^(1/p)` of the check-plot model; higher is better), and the number of pairs of neighbouring check plots (same column in consecutive rows, or same row in consecutive columns) inside a block and across block boundaries.

design Complete plot-level design with row, column, block, entry, control flag, and plot order.

breedbase_design Breedbase-compatible columns: `plots`, `block`, `all_entries`, `rep`, and `is_control`.

References

Piepho, H.-P. and Williams, E. R. (2016). Augmented row-column designs for a small number of checks. Agronomy Journal 108, 2256-2262. doi:10.2134/agronj2016.06.0325

Examples

```
treatments <- paste0("G", seq_len(108))
controls <- paste0("Check", seq_len(4))

design <- augmented_row_column_design(
  treatments = treatments,
  controls = controls,
  rows_in_field = 12,
  cols_in_field = 12,
  rows_per_block = 4,
  cols_per_block = 4,
  n_candidates = 50,
  seed = 123
)

head(design$design)
design$summary

# Example 3 of Piepho and Williams (2016): 173 entries, 4 checks, unequal
# row and column groups, and 3 to 5 check plots per block.
pb <- rbind(c(4, 5, 4), c(3, 5, 3), c(3, 5, 4), c(3, 5, 3))
ex3 <- augmented_row_column_design(
  treatments = paste0("E", 1:173),
  controls = c("A", "B", "C", "D"),
  rows_per_rowgroup = c(6, 5, 6, 5),
  cols_per_colgroup = c(3, 4, 3),
  checks_per_block = pb,
  n_candidates = 3,
  seed = 1
)
ex3$summary[, c("method", "check_plots", "entry_plots", "error_df", "d_criterion")]
```

---

augmented_row_column_layouts

List the analyzable layouts of an augmented row-column field

Description

Tries every combination of block size and number of checks for a field and returns the layouts that can be analysed, so a layout can be chosen without writing a loop over `augmented_row_column_capacity()`. Each block gets one plot of every check. Suggested blocks have at least as many rows and at least as many columns as there are checks, leaving room for each check to occupy a distinct row and column.

Usage

```
augmented_row_column_layouts(
  rows_in_field,
```

```
  cols_in_field,
  rows_per_block = NULL,
  cols_per_block = NULL,
  n_controls = 2:6,
  min_error_df = 1,
  n_entries = NULL
)
```

Arguments

rows_in_field, cols_in_field	Field size.
rows_per_block, cols_per_block	Block sizes to try. By default every divisor of the field size from 2 up to the field size.
n_controls	Numbers of checks to try.
min_error_df	Smallest error df to keep.
n_entries	Optional number of entries to plant. Only layouts with at least that many entry plots are kept, and spare_plots shows how many entry plots would be left over.

Details

There is a trade-off: more check plots mean fewer entries but more error degrees of freedom, i.e. more reliable F-tests and spatial adjustment. Aim for at least about 10 error df when the field allows it (Clarke and Stefanova, 2011, recommend about 50 for large trials).

Value

A tibble with one row per analyzable layout, sorted by entry plots (most first) and then error df: block sizes, number of checks, blocks, entry and check plots, check_fraction, error_df, method (the construction that augmented_row_column_design() uses for it: "latin" or "optimal") and, when n_entries is given, spare_plots.

See Also

augmented_row_column_capacity(), augmented_row_column_design()

Examples

```
# Suggested analyzable layouts of a 12 x 12 field with 2 to 5 checks:
augmented_row_column_layouts(12, 12, n_controls = 2:5)

# Layouts that fit 100 entries with at least 10 error df:
augmented_row_column_layouts(12, 12, n_controls = 2:6, min_error_df = 10, n_entries = 100)
```

`breedbase_adjusted_trait` *Find or create the Breedbase trait for adjusted values*

Description

First-stage values need their own trait, so raw and adjusted values stay apart. Breedbase makes such traits by composing the measured trait with a statistic term, e.g. "fresh root yield" + "Adjusted Means from Spatial Correction using SpATS R" gives the composed trait `"fresh root yield|Adjusted Means from Spatial Correction using SpATS R"` (ontology `COMP`). This function finds that trait, or creates it with `create = TRUE` through the same calls as the "Compose a new trait" page of Breedbase.

Usage

```
breedbase_adjusted_trait(con, traits, statistic = NULL, create = FALSE)
```

Arguments

`con`	A connection from `breedbase_login()`.
`traits`	Measured trait names, as in Breedbase.
`statistic`	The statistic term, with or without its id, as listed by `breedbase_statistics()`. By default it is chosen from this Breedbase's own list of statistics: the spatial correction term (SpATS). It is always checked against the server.
`create`	Logical. Create the traits that do not exist yet.

Value

A tibble with `trait`, `adjusted_trait`, `adjusted_trait_db_id` and `status` (`"exists"`, `"created"` or `"missing"`).

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
breedbase_adjusted_trait(con, "fresh root yield")                  # look only
breedbase_adjusted_trait(con, "fresh root yield", create = TRUE)   # create if needed

## End(Not run)
```

`breedbase_analysis_plots` *Plots of an analysis saved in Breedbase*

Description

Reads back the plots of an existing analysis (from `save_breedbase_analysis()` or made on the website): the analysis plot names, the field plot (or accession) each one stands for, its accession and design, and the values of every trait stored in the analysis. Use it to see what an analysis holds before adding values with `add_breedbase_analysis_values()`, e.g. a second trait.

Usage

```
breedbase_analysis_plots(con, analysis_name)
```

Arguments

`con`	A connection from `breedbase_login()`.
`analysis_name`	Name of the analysis.

Details

Analysis plots are not part of BrAPI, so they are read through the web pages of Breedbase (the site's web password is needed when it has one; see `breedbase_login()`).

Breedbase lists the plots of an analysis through their values: a plot without any value (e.g. a lost plot, or an outlier removed before the analysis) is not listed, although it is part of the analysis.

Value

A tibble with `analysis_plot_name`, `plot_name` (the field plot, or the accession for accession analyses: the name to use in the `plot` or `accession` column of `add_breedbase_analysis_values()`), `accession`, `plot_number`, `rep`, `block`, `row`, `col` and one column per trait, ordered by plot number. The analysis id is in the attribute `"analysis_db_id"`.

Examples

```
## Not run:
con <- breedbase_login("https://cassava-test.sgn.cornell.edu", "jdoe", site_username = "sgn")
plots <- breedbase_analysis_plots(con, "AYT_2026_stage1")
head(plots)

## End(Not run)
```

`breedbase_check` *Check names in Breedbase before building a design*

Description

Run this first, before designing the trial: it checks that the location(s) and every entry and check are already in Breedbase (and, when given, the breeding program, the folder and that the trial name is free). All problems are reported together, with close matches as suggestions (for example `"ibadan"` -> `"Ibadan"`), so they can be fixed in one go. `save_breedbase()` repeats the same checks before writing.

Usage

```
breedbase_check(
  con,
  accessions = NULL,
  location = NULL,
  program = NULL,
  folder = NULL,
  trial_name = NULL,
  plot_names = NULL
)
```

Arguments

`con`	A connection from `breedbase_login()`.
`accessions`	Entry and check names: a character vector, a design from `augmented_row_column_design()`, `augmented_prep_design()`, `resolvable_row_column_design()` or `as_breedbase_design()`, or a list of such designs (e.g. one per location).
`location`	Breedbase location name(s).
`program`, `folder`	Optional breeding program and folder (the folder is optional for saving too: without it the trial goes directly in the program).
`trial_name`	Optional trial name(s), e.g. one per location. A name already used by a trial (with plots) is reported as `"TAKEN"`: change it before saving.
`plot_names`	Optional plot names to check: a character vector, a design, or a list of designs. By default the plot names stored in the design(s) given in `accessions` are checked (see `add_plot_names()`). A design without stored plot names gets the default names of `save_breedbase()` for the check when it can be matched to one trial name and location (one design and one `trial_name`, or a list of designs and one trial name and location per design). Plot names must be unique in the whole database: a name already used is reported as `"TAKEN"`, a name given to two plots as `"REPEATED"`. To save with other names, change the `plot_name` column of the design or use `add_plot_names()`.

Value

Invisibly, a tibble with one row per name checked: `what`, `name`, `status` (`"OK"`, `"MISSING"`, `"NEW"`, `"TAKEN"`, `"EMPTY"` or `"REPEATED"`) and `suggestion`. Its attribute `"ok"` is `TRUE` when nothing needs fixing.

Slow servers

Accessions are searched 25 names at a time, because Breedbase looks up the pedigree of each one. If the server times out (HTTP 504, 502, 503 or 429), the batch is split in half and tried again, so a busy server is slower but still works. A line is printed after each batch with the names done and the time. Accessions found are remembered until R is restarted, so `save_breedbase()` after `breedbase_check()` does not search them again.

When creating the folder, the trial or its plots times out, the save does not stop: the server may still finish the job, so it checks for the result every 20 seconds for 3 minutes, and tries once more if it is not there (Breedbase refuses names that already exist, so this cannot make duplicates). If a save is cut off anyway, run `save_breedbase()` again: an existing folder is used, and a trial left without plots gets its plots. Settings: `options(breedtools.write_wait = 30, breedtools.write_checks = 10)`. Settings: `options(breedtools.germplasm_batch = 10)` for smaller batches, `options(breedtools.breedbase_timeout = 600)` to wait longer (seconds) for each answer.

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
entries <- c(paste0("IITA-TMS-IBA", 1:108))
checks <- c("TMEB419", "IITA-TMS-IBA30572", "TMEB693", "IITA-TMS-IBA000070")
breedbase_check(con, accessions = c(entries, checks), location = "Ibadan",
                program = "IITA", folder = "AYT 2026", trial_name = "AYT_2026_Ibadan")

## End(Not run)
```

`breedbase_lists` *List breeding programs, folders and locations in Breedbase*

Description

Use these to find the exact names to give to `save_breedbase()`.

Usage

```
breedbase_programs(con)

breedbase_folders(con, program)

breedbase_locations(con)
```

Arguments

con	A connection from breedbase_login().
program	For breedbase_folders(): the breeding program name.

Value

A tibble with names and database ids.

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
breedbase_programs(con)
breedbase_folders(con, program = "IITA")
breedbase_locations(con)

## End(Not run)
```

---

breedbase_login *Log in to a Breedbase server*

Description

Logs in through the BrAPI token call and returns a connection used by the other breedbase_*() functions and by save_breedbase(). Only the access token is kept; the password is not stored.

Usage

```
breedbase_login(
  url,
  username,
  password = NULL,
  site_username = NULL,
  site_password = NULL
)
```

Arguments

url	Address of the Breedbase site, e.g. "https://cassavabase.org".
username	Your Breedbase username.
password	Optional password; see Details.
site_username, site_password	Only for sites with an extra web password in front of the website (test servers such as cassava-test.sgn.cornell.edu ask for it in a browser pop-up, "Restricted Test Site"): that user name and password. They are used for the non-BrAPI calls (trait composer, analyses); BrAPI does not need them. Usually not needed:

when a site asks for a web password, the login first tries your Breedbase user name and password there, and uses them if they work. Leave `site_password` empty to read it from `BREEDBASE_SITE_PASSWORD` or to be asked in a pop-up.

Details

Never write your password in a script. Leave `password` empty: it is read from the environment variable `BREEDBASE_PASSWORD` if set (for example in your `.Renviron` file), and otherwise you are asked for it in a pop-up.

Value

A `breedbase_connection` object.

See Also

`save_breedbase()`, `breedbase_programs()`, `breedbase_folders()`, `breedbase_locations()`

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", username = "jdoe")
con

## End(Not run)
```

---

breedbase_phenotypes *Read phenotypic data of trials from Breedbase*

Description

Reads, through BrAPI, the plots of the given trials (plot name and number, accession, rep, block, row, column, check) and their values of the given traits, and returns one table with one row per plot and one column per trait, ready for `plot_trait()`, `remove_outliers()`, `analyze_two_stage()` and `analyze_single_stage()`. Repeated measurements of a trait on a plot are averaged (by Breedbase).

Usage

```
breedbase_phenotypes(con, trials, traits)
```

Arguments

`con`	A connection from `breedbase_login()`.
`trials`	Trial names, as in Breedbase.
`traits`	Trait names, as in Breedbase, with or without the ontology id (e.g. `"fresh root yield"` or `"fresh root yield|CO_334:0000013"`).

Value

A tibble: `trial_name`, `location`, `year`, `plot_name`, `plots` (plot number), `accession`, `rep`, `block`, `row`, `col`, `is_check` and one numeric column per trait (named as given in `traits`). The trait names in Breedbase are in `attr(result, "trait_names")`.

Examples

```
## Not run:
con <- breedbase_login("https://cassava-test.sgn.cornell.edu", "jdoe", site_username = "sgn")
pheno <- breedbase_phenotypes(con, trials = c("FALL_2026_Ibadan", "FALL_2026_Ikenne"),
                          traits = c("fresh root yield", "dry matter content percentage"))
plot_trait(pheno, "fresh root yield", by = "trial_name")

## End(Not run)
```

---

breedbase_refresh *Refresh the Breedbase phenotype cache*

Description

Values saved through BrAPI (`save_breedbase_phenotypes()`, `save_breedbase_first_stage()`) are in the database at once (the BrAPI search finds them), but the trial pages and the search wizard of Breedbase read a cache ("materialized views") that the BrAPI upload does not refresh. This asks Breedbase to refresh it, like the "Update" button of the search wizard. The save functions call it after saving. If it cannot be reached, the values are still saved: they appear after the next refresh of the site (usually overnight), or when you click "Update" on the Search Wizard page.

Usage

```
breedbase_refresh(con, views = c("phenotypes", "fullview"), quiet = FALSE)
```

Arguments

`con`	A connection from `breedbase_login()`.
`views`	Which cache to refresh: `"phenotypes"` (default) or `"fullview"`.
`quiet`	Logical. If `FALSE`, print what happened.

Value

Invisibly, `TRUE` if Breedbase accepted the refresh, else `FALSE`.

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
breedbase_refresh(con)

## End(Not run)
```

`breedbase_statistics` *List the statistic terms of Breedbase (SGNSTAT ontology)*

Description

Breedbase saves analysis results as a trait combined with a statistic term of its statistics ontology (SGNSTAT), e.g. "Adjusted Means from Spatial Correction using SpATS R". Breedbase loads these terms from the file `ontology/cxgn_statistics.obo` of its source code, which breedtools includes (`system.file("extdata", "cxgn_statistics.obo", package = "breedtools")`), so the terms can be listed and checked without a server. With a connection, the terms of that Breedbase are listed and compared with the ontology file. When Breedbase adds statistics, the bundled copy is refreshed with the script `data-raw/update_statistics_ontology.R` of the package source.

Usage

```
breedbase_statistics(con = NULL, pattern = "")
```

Arguments

`con`	Optional connection from `breedbase_login()`. Without it, the terms of the ontology file are listed.
`pattern`	Optional text the term must contain (any case).

Value

A tibble with `statistic` (the name), `accession` (e.g. `"SGNSTAT:0000054"`) and `definition` (from the ontology file); with a connection also `db_id` (the server's id) and `in_ontology` (`TRUE` when name and accession match the ontology file).

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
breedbase_statistics(con)
breedbase_statistics(con, "BLUE")

## End(Not run)
breedbase_statistics(pattern = "spatial")      # from the ontology file
```

breedbase_trial_plots *Read the plots of a trial saved in Breedbase*

Description

Returns the plots of a Breedbase trial (through BrAPI): plot name, number, accession, rep, block, row and column. Use it to check that a design in R is the one saved in Breedbase (see `breedbase_plots` in `first_stage_values()`), or to work with a trial made elsewhere.

Usage

```
breedbase_trial_plots(con, trial_name)
```

Arguments

`con`	A connection from `breedbase_login()`.
`trial_name`	The trial name in Breedbase.

Value

A tibble with `plot_name`, `plot_number`, `accession`, `rep`, `block`, `row`, `col` and `is_check`, ordered by plot number.

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
plots <- breedbase_trial_plots(con, "AYT_2026_Ibadan")

## End(Not run)
```

calculate_environment_stratification_anova *Calculate design-aware ANOVA*

Description

Calculate design-aware ANOVA

Usage

```
calculate_environment_stratification_anova(data)
```

Arguments

`data`	Standardized phenotype data.

Value

ANOVA table as a data frame.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

calculate_environment_stratification_anova(prepared$data)
```

---

check_design *Standard checks of a design*

Description

Runs the structural checks that every design of its kind should pass and prints one [OK] / [FAIL] line per check ([NOTE] for informational ones). The results are recorded for check_summary().

Usage

```
check_design(design, section = NULL)
```

Arguments

design	A design object from augmented_row_column_design(), augmented_prep_design() or resolvable_row_column_design().
section	Optional label for the recorded checks. Defaults to the label set with check_section(), or else the kind of design.

Details

Augmented row-column designs every plot used once, every entry planted once, every field row and column has a check, the requested number of check plots in each block, positive error df, all row, column and block effects estimable from the checks, and no check plots next to each other inside a block (a note for neighbours across block boundaries).

Augmented p-rep designs every entry at every location, no entry twice in a block, block sizes within a location differ by at most one, every location connected, the pairwise concurrence (a note when pairs share more than one block), and the replicate structure of resolvable designs.

Resolvable row-column designs every treatment once in every replicate, every plot used once, the spatial efficiency below its upper bound, and latinization when requested.

Value

Invisibly, TRUE if all (non-note) checks passed.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
check_reset()
check_design(rc)
check_summary()
check_reset()
```

---

check_reset *Clear the recorded checks*

Description

Removes the checks recorded so far with `check_result()` and `check_design()`, so that the next `check_summary()` only counts new checks. It does not check or change any design or data; it only empties the list. A message says how many earlier checks were cleared and how many of them had passed.

Usage

```
check_reset()
```

Value

Invisibly, a tibble of the checks that were cleared.

Examples

```
check_reset()
check_result("one equals one", 1 == 1)
check_reset()
```

---

check_result *Record and print a check*

Description

Prints `[OK]` or `[FAIL]` followed by a description, and records the result so that `check_summary()` can report how many checks passed. Use it to write your own checks while exploring a design or an analysis.

Usage

```
check_result(what, ok, section = NULL, note = FALSE)
```

Arguments

what	Description of the check, e.g. "every entry planted once".
ok	Logical. TRUE when the check passes; anything else (including NA) counts as a failure.
section	Optional label for grouping checks, e.g. "p-rep". Defaults to the label set with check_section().
note	Logical. If TRUE, the check is informational: it prints [NOTE] instead of [FAIL] when ok is not TRUE and is not counted as a failure.

Value

Invisibly, TRUE if the check passed.

See Also

check_section(), check_summary(), check_reset(), check_design()

Examples

```
check_reset()
check_result("two plus two is four", 2 + 2 == 4)
check_result("a deliberate failure", FALSE, section = "demo")
check_summary()
check_reset()
```

---

check_section *Set the section label for the next checks*

Description

Checks recorded after this call with check_result() and check_design() are grouped under name in check_summary().

Usage

```
check_section(name)
```

Arguments

name	Section label, e.g. "1. Row-column design". NULL removes the label.

Value

Invisibly, the previous label.

Examples

```
check_reset()
check_section("arithmetic")
check_result("one equals one", 1 == 1)
check_summary()
check_reset()
```

`check_setup` *Check that everything is ready before starting*

Description

Run this at the start of a script. It starts a fresh list of checks (so the final `check_summary()` only counts this run) and checks that you are ready to design and analyse trials:

- breedtools is loaded, and which version;
- the optional packages 'lme4' (mixed models with random blocks, random entries or random genotype-by-location) and 'SpATS' (spatial models) are installed; a missing one is a note, not a failure, and the message says what it is needed for;
- the folder for your files exists (it is created if needed) and files can be written to it.

Usage

```
check_setup(out_dir = NULL)
```

Arguments

`out_dir`	Optional folder where the script will write field books and other files.

Value

Invisibly, `TRUE` if everything needed is in place.

See Also

`check_summary()`, `check_design()`, `check_result()`

Examples

```
check_setup(out_dir = file.path(tempdir(), "my_trials"))
```

check_summary *Summary of the recorded checks*

Description

Reports what was checked and what is correct: how many of the checks recorded with `check_result()` and `check_design()` passed, broken down by section, followed by the checks that failed and the notes.

Usage

```
check_summary(details = FALSE, print = TRUE)
```

Arguments

`details`	Logical. If `TRUE`, also list every check with its status.
`print`	Logical. If `TRUE` (default), print the report.

Value

Invisibly, a tibble of all recorded checks with columns `section`, `check` and `status` (`"OK"`, `"FAIL"` or `"NOTE"`).

Examples

```
check_reset()
check_result("one equals one", 1 == 1, section = "arithmetic")
check_result("two is even", 2 %% 2 == 0, section = "arithmetic")
check_summary()
check_summary(details = TRUE)
check_reset()
```

compare_first_stage *Compare raw and first-stage adjusted values*

Description

Puts each plot's raw value next to its first-stage value (the genotype's adjusted mean plus the plot's residual, as in `first_stage_values()`), summarises how much the first stage changed (per location: correlation of raw and adjusted plot values, the spread before and after, and how much the ranking of the genotypes changed) and draws, per location, the raw and the adjusted field maps and a raw-versus-adjusted plot.

Usage

```
compare_first_stage(fit, plot = TRUE, locations = NULL)
```

Arguments

fit	An analysis (e.g. analyze_augmented_row_column_design() with spatial = TRUE) or a two-stage analysis (analyze_two_stage(), analyze_augmented_prep_two_stage()), whose stage-1 fits are compared.
plot	Logical. Draw the plots.
locations	For two-stage analyses: the locations to draw (default all).

Value

Invisibly, a list with plots (one row per plot: location, plot, genotype, row, col, raw, adjusted, change) and summary (one row per location). The summary is also printed.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
d <- rc$design
set.seed(1)
d$yield <- 50 + 0.8 * d$row + rnorm(nrow(d), sd = 2)
fit <- analyze_augmented_row_column_design(d, "yield")
cmp <- compare_first_stage(fit)
```

---

complete_environment_info

Complete environment metadata

Description

Complete environment metadata

Usage

```
complete_environment_info(env_info)
```

Arguments

env_info	Data frame with environment, location, trial, and year.

Value

Data frame with an added environment_label column.

Examples

```
env_info <- data.frame(
  environment = c("Loc1_T1", "Loc2_T2"),
  location = c("Loc1", "Loc2"),
  trial = c("T1", "T2"),
  year = c("2025", NA)
)

complete_environment_info(env_info)
```

`design_options` *Suggest trial layouts from the number of entries and available field space*

Description

Lists parameter options for the package's three design families, starting with the information the breeder knows. Rows, columns and check counts can be left unspecified. No randomization or optimization is performed.

Usage

```
design_options(
  n_entries,
  design = c("augmented_row_column", "prep", "resolvable"),
  n_checks = NULL,
  rows_in_field = NULL,
  cols_in_field = NULL,
  n_locations = 1L,
  n_reps = 2:3,
  block_size = c(5L, 8L, 10L, 12L, 16L, 20L),
  prep_prop = NULL,
  max_plots = NULL,
  max_dimension = NULL,
  min_error_df = NULL,
  n_options = 6L,
  n_candidates = 200L,
  rows = NULL,
  cols = NULL
)
```

Arguments

`n_entries`	Number of test entries, excluding checks.
`design`	Design families to consider: `"augmented_row_column"` (augmented row-column), `"prep"` (partially replicated), and `"resolvable"` (fully replicated row-column). Defaults to all three.

`n_checks` Distinct check genotypes to consider, a non-negative integer or vector. `NULL` tries 2 through 6 for augmented row-column designs and zero for the other families. p-rep uses no separate checks; it is excluded when only positive check counts are requested. Resolvable designs replicate checks just like the other genotypes.

`rows_in_field, cols_in_field` Known dimensions of the whole field at one location (plot rows and plot columns, including all checks and replicates), not of a block. Block dimensions are suggested in the result as `rows_per_block` and `cols_per_block`. Each can be a positive integer, a vector of dimensions to consider, or `NULL` to search automatically. Supplied dimensions are exact candidate sizes, not upper bounds. When only one is supplied, the square field (the same number of rows and columns) is also considered. For p-rep, the last row may be partially occupied, but wholly empty final rows are not proposed.

`n_locations` Number of locations. Augmented row-column and resolvable layouts describe one field to randomize independently at each location. A p-rep option describes the combined multi-location design.

`n_reps` Full replication counts to try for resolvable designs (at least 2). Other design families use their own replication scheme.

`block_size` Maximum incomplete block sizes to try for p-rep.

`prep_prop` Proportions of entries to duplicate at each p-rep location. Counts use `round(n_entries * prep_prop)`, as in the generator. An entry may be duplicated at only one location, so the sum of these counts over locations must not exceed `n_entries`. `NULL` tries 15%, 20% and 25%, and, for several locations, a balanced share of entries per location rounded down to respect this budget.

`max_plots` Largest field area (rows times columns) per location. Defaults to four times the number of genotypes (increased when more full replicates are requested), or the largest supplied field area when both dimensions are given.

`max_dimension` Largest automatically searched row or column dimension. Defaults to twice the square root of `max_plots`, with a minimum of 20. Explicit dimensions are not restricted by this bound. When neither dimension is supplied, fields with an aspect ratio above 3 are excluded. Supply a dimension explicitly to consider longer fields.

`min_error_df` Minimum nominal error degrees of freedom. These are count-based values for the design model, assuming its effects are estimable, not degrees of freedom of a later spatial model. `NULL` (default) requires at least 1, except for the layout of Fig. 2 of Piepho and Williams (2016): blocks of `n_checks` x `n_checks` plots with each check exactly once, such as 36 entries and three checks in a 9 x 6 field. That layout is saturated with fixed rows, columns and blocks (error df = 0) and is analyzed with `analyze_augmented_row_column_design(random_blocks = TRUE)` or `entries = "random"`. A supplied value applies to every layout; 0 also admits other saturated layouts.

`n_options` Maximum options returned per design family. The shortlist includes different check, full-replication or p-rep counts before adding further layouts with the same counts.

`n_candidates` Search candidates to pass to the augmented row-column and p-rep generators. Defaults to 200. This is a user-selected search effort, not a dimension inferred from entry counts.

rows, cols Deprecated names of rows_in_field and cols_in_field.

Details

This is a parameter planner. Candidates pass plot-count, block-size and nominal error-df checks. Augmented row-column candidates also pass the check coverage and row/column group conditions used by the generator. p-rep candidates have enough repeated plots to connect their blocks and a pair-count budget compatible with (0,1) concurrence. These necessary conditions do not certify a randomized design's connectivity or efficiency: generate the chosen option, inspect its summary and run check_design(). The search is bounded and not exhaustive.

Augmented row-column options with equal row and column groups have blocks of at least n_checks rows and at least n_checks columns, leaving room for one copy of each check in a distinct row and column. For example, four checks require blocks of at least 4 rows by 4 columns. Every such block contains all checks at least once; extra positions are assigned to extra check replication, so the number of test-entry positions is exactly n_entries. Flexible allocations use method = "optimal".

Options with unequal row or column groups (Piepho and Williams, 2016, Example 3 and Fig. 4) split the field into near-equal groups, such as rows 6, 5, 6, 5 and columns 3, 4, 3 for a 22 x 10 field. Their blocks hold n_checks - 1 to n_checks + 1 check plots, in proportion to block size, so some blocks may be incomplete for the checks. These options report rows_per_rowgroup, cols_per_colgroup and checks_per_block, with rows_per_block and cols_per_block set to NA.

Resolvable options may contain filler positions, which are reported in unused_plots. Include both entries and checks in the generator's treatments. Its native generator treats them all as treatments; if checks need to be marked in Breedbase, set is_control in both the generated design and breedbase_design tables before saving.

Square blocks are preferred but not required: for augmented row-column designs, when a layout with blocks of n_checks rows by n_checks columns containing each check exactly once exists (for example 3 x 3 blocks with checks A, B and C, as in Fig. 2 of Piepho and Williams, 2016), the smallest one is the first option and square_check_blocks flags it. Blocks of that size with extra check plots are not flagged. The other options follow in the usual order: within each resource-count group, smaller field areas and fewer unused positions come first. Within a field, augmented row-column layouts whose blocks hold about one check plot per check (complete blocks for the checks, as in Piepho and Williams, 2016) come first, then Latin constructions, squarer blocks and less elongated fields. This ordering does not estimate statistical precision or choose the biologically best design.

Value

A tibble with option id, design family, entry/check counts, field dimensions, a square_check_blocks flag, location and replication counts, active plots per location and in total, field area, unused positions, check plots, repeated entries, blocks, nominal error_df, method and notes. Field dimensions are named rows_in_field and cols_in_field, matching augmented_row_column_design(). Block and replicate dimensions are included where applicable. n_candidates reports the chosen search effort. checks_per_block is a list-column with each augmented row-column option's check-allocation matrix, or NULL for the default complete check blocks. These columns support an explicit generator call. rows_per_rowgroup and cols_per_colgroup are list-columns with the group sizes of unequal-group options, or NULL. generator names the function to call; arguments

is a list-column of its layout arguments. Add your own `treatments`, `controls` (augmented row-column) and `seed` before calling it with `do.call()`. p-rep arguments include placeholder location names, which can be replaced with your own names. Attribute `diagnostics` reports the number of shortlisted options and the reason for any excluded family. No matches return a typed empty tibble and a message suggesting which constraints to relax.

See Also

augmented_row_column_layouts(), augmented_prep_parameters(), augmented_row_column_design(), augmented_prep_design(), resolvable_row_column_design()

Examples

```
# Only the number of entries is known:
options <- design_options(100)
options[, c("option", "design", "n_checks", "rows_in_field", "cols_in_field", "plots_per_location")]

# A known field, four checks and exactly 108 test entries:
options <- design_options(108, design = "augmented_row_column", n_checks = 4,
                          rows_in_field = 12, cols_in_field = 12, n_candidates = 5)
selected <- options[1, ]
trial <- augmented_row_column_design(
  treatments = paste0("LINE_", seq_len(108)),
  controls = paste0("CHECK_", seq_len(selected$n_checks)),
  rows_in_field = selected$rows_in_field,
  cols_in_field = selected$cols_in_field,
  rows_per_block = selected$rows_per_block,
  cols_per_block = selected$cols_per_block,
  checks_per_block = selected$checks_per_block[[1]],
  method = selected$method,
  n_candidates = selected$n_candidates,
  seed = 2026
)

# Fully replicated options with one known dimension:
design_options(42, design = "resolvable", cols_in_field = 7, n_reps = 3)

# Only the number of field rows is known. With three checks, the first
# option is the layout of Piepho and Williams (2016, Fig. 2): a 9 x 6 field
# of six 3 x 3 blocks, each check once per block (error df = 0).
design_options(36, design = "augmented_row_column", n_checks = 3, rows_in_field = 9)
# Require positive error df for every layout:
design_options(36, design = "augmented_row_column", n_checks = 3, rows_in_field = 9,
               min_error_df = 1)
# Partial replication at three locations:
design_options(100, design = "prep", n_locations = 3)
```

`detect_design` *Detect field trial design for environment stratification*

Description

Detects whether the data look like CRD, RCBD, row-column, alpha-lattice, or augmented/block-only design.

Usage

```
detect_design(data)
```

Arguments

data	Standardized phenotype data.

Value

A list with design, formula, and message.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

design <- detect_design(prepared$data)
design$design
design$formula
```

---

environment_summary_by_group

Summarize environment names by group

Description

Summarize environment names by group

Usage

```
environment_summary_by_group(group_summary, group_membership)
```

Arguments

group_summary	Group summary table.
group_membership	Group membership table.

Value

Group summary with display metadata.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

lin <- lin_group_environments(prepared$data)
membership <- add_environment_metadata(lin$group_membership, prepared$env_info)
environment_summary_by_group(lin$group_summary, membership)
```

`find_outliers` *Find outliers in trait data (IQR rule)*

Description

Flags values below $Q_1 - k \cdot IQR$ or above $Q_3 + k \cdot IQR$, where Q_1, Q_3 are the quartiles and k is `multiplier` (Tukey's fences; 1.5 by default, 3 for only the extreme values). The rule is applied within each group of `by`, usually each location, because locations have different means.

Usage

```
find_outliers(
  data,
  traits,
  multiplier = 1.5,
  by = if ("location" %in% names(data)) "location" else NULL,
  id_cols = c("location", "plot_name", "plots", "plot_number", "all_entries",
    "accession", "row", "col")
)
```

Arguments

`data`	A data frame with one row per plot.
`traits`	Names of the numeric trait columns.
`multiplier`	The IQR multiplier k.
`by`	Optional grouping column(s); by default `"location"` when the data have it.
`id_cols`	Columns that identify the plot in the report (those present in `data`).

Value

A tibble with one row per outlier: the identifying columns, the group, `trait`, `value`, the fences `lower` and `upper`, and `side` (`"low"` or `"high"`).

Examples

```
d <- data.frame(plots = 1:20, all_entries = paste0("G", 1:20),
                yield = c(rnorm(18, 50, 2), 80, 10))
find_outliers(d, "yield")
```

`first_stage_values` *First-stage plot values from an analysis*

Description

Turns analyses of a trial into one value per field plot, ready for `save_breedbase_first_stage()`: the genotype's adjusted mean plus the plot's residual. This removes the design effects (blocks, rows, columns, replicates) and the spatial trend, and keeps the plot-to-plot variation, so the result has as many plots as the field trial. With SpATS (`spatial = TRUE` or `stage1 = "spatial"`) these are spatially corrected plot values.

Usage

```
first_stage_values(
  fits,
  design = NULL,
  trial_name = NULL,
  trait = NULL,
  plot_name_pattern = NULL,
  plot_names = NULL,
  breedbase_plots = NULL,
  location = NULL
)
```

Arguments

`fits`	One analysis, or a named list of analyses, one per trait (the names are the Breedbase trait names). Analyses from `analyze_augmented_row_column_design()`, `analyze_augmented_prep_design()`, `analyze_augmented_prep_two_stage()` (stage-1 fits per location) or `analyze_resolvable_row_column()`.
`design`	The design that was saved to Breedbase. Leave it `NULL` for data read from Breedbase (`breedbase_phenotypes()`): the plot names are then taken from the data.
`trial_name`	The trial name used when saving the design (not needed without a design).
`trait`	Trait name when `fits` is a single analysis.
`plot_name_pattern`, `plot_names`, `location`	The plot naming used in `save_breedbase()`; `location` is the Breedbase location of the trial, needed for the `{location}` field when the design has no stored plot names.
`breedbase_plots`	Optional plots of the saved trial, from `breedbase_trial_plots()`. The design is then checked against the saved trial (the same accession at every plot number), and the plot names are taken exactly as saved.

Value

A tibble with `plot_name`, `trial_name`, `accession`, `plot_number`, `block`, `rep`, `row`, `col`, `is_check` and one column per trait. Its attribute `"method"` is `"spatial"` when any analysis used SpATS, otherwise `"mixed"`.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
d <- rc$design
set.seed(1)
d$yield <- 25 + 0.4 * d$row + rnorm(nrow(d), sd = 2)
fit <- analyze_augmented_row_column_design(d, "yield")
values <- first_stage_values(fit, rc, "AYT_2026_Ibadan", trait = "fresh root yield",
                             location = "Ibadan")
head(values)
```

`get_compatible_groups` *Get compatible environment groups*

Description

Get compatible environment groups

Usage

```
get_compatible_groups(group_tests)
```

Arguments

`group_tests`	Output from `make_environment_groups()`.

Value

A tibble containing only homogeneous groups.

Examples

```
env_anova <- data.frame(
  environment = c("Loc1", "Loc2", "Loc3"),
  ms_error = c(10, 12, 30),
  df_error = c(20, 20, 20)
)

group_tests <- make_environment_groups(env_anova, n_sim = 2000)
get_compatible_groups(group_tests)
```

get_maximal_groups *Select maximal compatible environment groups*

Description

Removes smaller compatible groups that are fully contained inside larger compatible groups.

Usage

```
get_maximal_groups(compatible_groups)
```

Arguments

compatible_groups	Output from get_compatible_groups().

Value

A tibble with maximal compatible groups only.

Examples

```
env_anova <- data.frame(
  environment = c("Loc1", "Loc2", "Loc3"),
  ms_error = c(10, 12, 30),
  df_error = c(20, 20, 20)
)

group_tests <- make_environment_groups(env_anova, n_sim = 2000)
get_maximal_groups(get_compatible_groups(group_tests))
```

guess_column *Guess a column name from possible Breedbase column names*

Description

Guess a column name from possible Breedbase column names

Usage

```
guess_column(df, possible_names)
```

Arguments

df	A data frame.
possible_names	A character vector of possible column names.

Value

A column name if found, otherwise NULL.

Examples

```
df <- data.frame(germplasmName = "G1", phenotype = 10)
guess_column(df, c("germplasmName", "accession_name"))
```

---

hartley_critical_sim *Simulate Hartley Fmax critical value*

Description

Estimates the Hartley Fmax critical value using simulation. This version can handle different residual degrees of freedom across environments.

Usage

```
hartley_critical_sim(df_error_vec, alpha = 0.05, n_sim = 1e+05, seed = 123)
```

Arguments

df_error_vec	Numeric vector of residual degrees of freedom.
alpha	Significance level. Default is 0.05.
n_sim	Number of simulations. Default is 100000.
seed	Random seed used for the simulation. The user's global random number generator state is restored on exit. Use NULL to draw from the current random number stream without setting a seed.

Value

A numeric critical value.

Examples

```
hartley_critical_sim(c(20, 20, 20), alpha = 0.05, n_sim = 2000)
```

hartley_fmax_test *Hartley Fmax test for residual variance homogeneity*

Description

Hartley Fmax test for residual variance homogeneity

Usage

```
hartley_fmax_test(env_anova, alpha = 0.05, n_sim = 1e+05, seed = 123)
```

Arguments

env_anova	A data frame containing environment-level ANOVA results. It must include columns environment, ms_error, and df_error.
alpha	Significance level. Default is 0.05.
n_sim	Number of simulations for the critical value.
seed	Random seed passed to hartley_critical_sim(). The user's global random number generator state is not modified.

Value

A tibble with observed Fmax, critical Fmax, and homogeneity decision.

Examples

```
env_anova <- data.frame(
  environment = c("Loc1", "Loc2", "Loc3"),
  ms_error = c(10, 12, 15),
  df_error = c(20, 20, 20)
)

hartley_fmax_test(env_anova, n_sim = 2000)
```

lin_error_mse *Calculate residual error MSE for a Lin environment group*

Description

Calculate residual error MSE for a Lin environment group

Usage

```
lin_error_mse(data, envs, common_genotypes)
```

Arguments

data	Standardized phenotype data.
envs	Character vector of environments.
common_genotypes	Character vector of common genotypes.

Value

A list with MSE, residual df, and message.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

envs <- prepared$env_info$environment[1:2]
genotypes <- unique(prepared$data$accession_name)
lin_error_mse(prepared$data, envs, genotypes)
```

---

lin_group_environments

Group environments using Lin-style GE compatibility testing

Description

Group environments using Lin-style GE compatibility testing

Usage

```
lin_group_environments(data, alpha = 0.05)
```

Arguments

data	Standardized phenotype data.
alpha	Significance level.

Value

A list with pairwise tests, group summary, group membership, and ungrouped environments.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

lin <- lin_group_environments(prepared$data, alpha = 0.05)
lin$group_summary
lin$group_membership
```

---

lin_ss_ge *Calculate Lin genotype-by-environment sum of squares*

Description

Calculate Lin genotype-by-environment sum of squares

Usage

```
lin_ss_ge(data, envs)
```

Arguments

data	Standardized phenotype data.
envs	Character vector of environments to test.

Value

A list with a summary table and common genotype names.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

envs <- prepared$env_info$environment[1:2]
lin_ss_ge(prepared$data, envs)$summary
```

`lin_test_group` *Test one Lin environment group*

Description

Test one Lin environment group

Usage

```
lin_test_group(data, envs, alpha = 0.05)
```

Arguments

`data`	Standardized phenotype data.
`envs`	Character vector of environments.
`alpha`	Significance level.

Value

Data frame with Lin GE test result.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

envs <- prepared$env_info$environment[1:2]
lin_test_group(prepared$data, envs, alpha = 0.05)
```

`make_environment_groups` *Test all compatible environment groups using Hartley Fmax*

Description

Tests all combinations of environments and identifies groups with homogeneous residual variances.

Usage

```
make_environment_groups(
  env_anova,
  alpha = 0.05,
  max_group_size = Inf,
  n_sim = 1e+05,
  seed = 123
)
```

Arguments

env_anova	A data frame from analyze_environment_anova().
alpha	Significance level.
max_group_size	Maximum environment group size to test.
n_sim	Number of simulations for Hartley critical value.
seed	Random seed.

Value

A tibble with all tested groups.

Examples

```
env_anova <- data.frame(
  environment = c("Loc1", "Loc2", "Loc3"),
  ms_error = c(10, 12, 30),
  df_error = c(20, 20, 20)
)

make_environment_groups(env_anova, n_sim = 2000)
```

---

map_environment_groups

Create a map of environment groups

Description

Creates an interactive leaflet map where locations belonging to the same environment group use the same marker color. Ungrouped environments are shown with ungrouped_color.

Usage

```
map_environment_groups(
  stratification_result,
  locations,
  ignore_case = FALSE,
  group_palette = NULL,
  ungrouped_color = "#7A7A7A",
  match_by = "location",
  strict = TRUE,
  tiles = "breedbase"
)
```

Arguments

stratification_result	Result from `run_environment_stratification()` or `run_breedbase_environment_stratificatior`
locations	Location coordinates from `read_locations()`, a location file path, or a data frame with location, latitude, and longitude columns.
ignore_case	Logical. If `TRUE`, location matching ignores case.
group_palette	Optional vector of colors used for environment groups.
ungrouped_color	Color for ungrouped environments.
match_by	Name from the stratification result used to match the `location` column in `locations`. Use `"location"` when coordinates are named by location, `"environment"` when coordinates are named by the constructed environment identifier, or `"environment_label"` when coordinates are named by display labels.
strict	Logical. If `TRUE`, stop when any environment location has no matching coordinates.
tiles	Background map. The default, `"breedbase"`, uses the same maps as the 'Breedbase' Locations page: Esri Street (shown first), Topographical and Satellite, with a switch in the top-right corner. They need no account or API key and work in the RStudio viewer and in saved HTML files. `"street"`, `"topo"` or `"satellite"` shows only that map. `"OpenStreetMap"` uses OpenStreetMap's own tiles (these may be refused outside a website). Any other value is passed to `leaflet::addProviderTiles()` as a provider name; some commercial providers need an API key.

Value

A `leaflet` htmlwidget.

Examples

```
pheno_file <- system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
loc_file <- system.file("extdata", "breedbase_locations.csv", package = "breedtools")

result <- run_environment_stratification(pheno_file, trait = "fresh root yield")
```

```
locations <- read_locations(loc_file, phenotype_data = result)

group_map <- map_environment_groups(result, locations)
if (interactive()) group_map
```

`phenotype_template` *Phenotyping template with the Breedbase plot names of a design*

Description

Makes a table with one row per plot, the plot names that `save_breedbase()` gives (or gave) to the plots, the design columns (accession, block, rep, row, column) and one empty column per trait. Fill it in (in R or in a spreadsheet) and upload it with `save_breedbase_phenotypes()`.

Usage

```
phenotype_template(
  design,
  trial_name,
  traits = character(0),
  plot_name_pattern = NULL,
  plot_names = NULL,
  location = NULL
)
```

Arguments

`design`	A design from `augmented_row_column_design()`, `augmented_prep_design()`, `resolvable_row_column_design()`, `as_breedbase_design()` or `prepare_breedbase_trial()`.
`trial_name`	The trial name used when saving the design. For p-rep designs saved as one trial per location, the location is added as in `save_breedbase()`. Can be omitted for a prepared Breedbase trial or a design with plot names (see `add_plot_names()`).
`traits`	Trait names as in Breedbase, e.g. `"fresh root yield"` or `"fresh root yield|CO_334:0000013"`. Can be empty (no trait columns).
`plot_name_pattern`, `plot_names`	The same plot naming used in `save_breedbase()`. By default the plot names stored in the design are used, when it has them.
`location`	The Breedbase location of the trial, needed when the plot names are made here with the `{location}` field (the default pattern) and the design has no stored plot names.

Value

A tibble: `plot_name`, `trial_name`, `accession`, `plot_number`, `block`, `rep`, `row`, `col`, `is_check` and one column per trait.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
sheet <- phenotype_template(rc, "AYT_2026", location = "Ibadan",
                            traits = c("fresh root yield", "dry matter content percentage"))
head(sheet)
```

`plot_augmented_prep_design` *Plot an augmented p-rep design grid*

Description

Draws one field map per location for an augmented p-rep design.

Usage

```
plot_augmented_prep_design(
  design,
  locations = NULL,
  label = c("all_entries", "entry_type", "repeated_at", "block", "replicate", "none"),
  fill = c("entry_type", "is_extra_repeat", "repeated_at", "block", "replicate"),
  show_plot_labels = TRUE,
  show_block_labels = FALSE,
  block_label_position = c("background", "center"),
  label_cex = NULL,
  block_label_cex = NULL,
  block_label_col = NULL,
  block_lwd = 2,
  main = NULL,
  legend = TRUE,
  caption = TRUE,
  ...
)
```

Arguments

`design`	An object returned by `augmented_prep_design()` or its `design` data frame.
`locations`	Optional character vector of locations to plot. Defaults to all locations in design order.
`label`	Cell label to show. Use `"all_entries"`, `"entry_type"`, `"repeated_at"`, `"block"`, `"replicate"`, or `"none"`.
`fill`	Cell coloring variable. Use `"entry_type"`, `"is_extra_repeat"`, `"repeated_at"`, `"block"`, or `"replicate"`.

`show_plot_labels`
: Logical. If `TRUE`, labels are printed inside cells unless `label = "none"`.

`show_block_labels`
: Logical. If `TRUE`, block numbers are printed. Defaults to `FALSE` so block labels do not overlap accession labels.

`block_label_position`
: Position for block labels when `show_block_labels = TRUE`. Use `"background"` for a faint watermark behind accession labels or `"center"` for a regular center label.

`label_cex`
: Optional text size for plot-cell labels.

`block_label_cex`
: Optional text size for block labels.

`block_label_col`
: Color for block labels.

`block_lwd`
: Line width for block boundaries.

`main`
: Optional plot title. If multiple locations are plotted, a single value is combined with the location name; a vector matching the number of locations is used panel by panel.

`legend`
: Logical. If `TRUE`, draw a legend for cell colors.

`caption`
: Logical. If `TRUE`, print a short explanation of the colours and lines under each map.

`...`
: Additional arguments passed to `graphics::plot.window()`.

Details

A p-rep design has no check plots: at each location some entries are planted more than once (repeated entries), and these replace the checks for estimating the error and the block effects. By default the map colours the repeated entries (orange, names in bold) and the entries with a single plot at that location (light grey). Thick black lines outline the incomplete blocks; in resolvable designs a thick blue line separates the replicates, which are labelled at the right. A caption under each map explains this.

Value

Invisibly returns the plotted design data frame with internal plotting coordinates.

Examples

```
design <- augmented_prep_design(
  treatments = paste0("G", seq_len(12)),
  locations = c("Loc1", "Loc2", "Loc3"),
  block_size = 5,
  n_candidates = 5,
  seed = 123,
  field_cols = 5
)

plot_augmented_prep_design(design)
```

plot_augmented_row_column_design
Plot an augmented row-column design grid

Description

Draws a field grid for an augmented row-column design, with plot cells, row and column labels, and thick outlines around row-column blocks.

Usage

```
plot_augmented_row_column_design(
  design,
  label = c("all_entries", "type", "block", "none"),
  fill = c("type", "block", "rowgroup", "colgroup", "is_control"),
  show_plot_labels = TRUE,
  show_block_labels = FALSE,
  block_label_position = c("background", "center"),
  label_cex = NULL,
  block_label_cex = NULL,
  block_label_col = NULL,
  block_lwd = 2,
  main = NULL,
  legend = TRUE,
  ...
)
```

Arguments

design	An object returned by augmented_row_column_design() or a design data frame with row, column, and block columns.
label	Cell label to show. Use "all_entries", "type", "block", or "none".
fill	Cell coloring variable. Use "type", "block", "rowgroup", "colgroup", or "is_control".
show_plot_labels	Logical. If TRUE, labels are printed inside cells unless label = "none".
show_block_labels	Logical. If TRUE, block numbers are printed. Defaults to FALSE so block labels do not overlap accession labels.
block_label_position	Position for block labels when show_block_labels = TRUE. Use "background" for a faint watermark behind accession labels or "center" for a regular center label.
label_cex	Optional text size for plot-cell labels.
block_label_cex	Optional text size for block labels.

block_label_col	Color for block labels.
block_lwd	Line width for block boundaries.
main	Optional plot title.
legend	Logical. If TRUE, draw a legend for cell colors.
...	Additional arguments passed to graphics::plot.window().

Value

Invisibly returns the plotted design data frame with internal plotting coordinates.

Examples

```
design <- augmented_row_column_design(
  treatments = paste0("G", seq_len(108)),
  controls = paste0("Check", seq_len(4)),
  rows_in_field = 12,
  cols_in_field = 12,
  rows_per_block = 4,
  cols_per_block = 4,
  n_candidates = 20,
  seed = 123
)

plot_augmented_row_column_design(design, label = "all_entries")
```

---

plot_resolvable_row_column_design

Plot a resolvable row-column design

Description

Draws the field with replicate boundaries.

Usage

```
plot_resolvable_row_column_design(
  design,
  label = c("all_entries", "replicate", "none"),
  fill = c("replicate", "is_filler"),
  main = NULL,
  ...
)
```

Arguments

design	An object from resolvable_row_column_design() or its design data frame.
label	Cell label: "all_entries", "replicate", or "none".
fill	Cell colouring: "replicate" or "is_filler".
main	Optional title.
...	Passed to plot_augmented_row_column_design().

Value

Invisibly, the plotted data.

Examples

```
design <- resolvable_row_column_design(paste0("T", 1:20), 2, 4, 5,
                                       n_starts = 1, n_iter = 300, seed = 1)
plot_resolvable_row_column_design(design)
```

---

plot_trait *Plot a trait: boxplot, distribution and field map*

Description

Draws the trait per location (or other group) so gradients, odd plots and outliers are easy to see:

"boxplot" All locations side by side. Outliers (IQR rule, as in find_outliers(), or the table given in outliers) are drawn in red and labelled with their plot number; the other plots in grey.

"histogram" The distribution at each location.

"field" A map of the field with the value of every plot, with a colour bar. The colour scale of each location spans the values inside the IQR fences, so a few extreme plots do not wash out the rest of the map; plots beyond the scale get the end colour and are circled in red. Missing plots are grey.

Usage

```
plot_trait(
  data,
  trait,
  by = if ("location" %in% names(data)) "location" else NULL,
  type = c("both", "histogram", "field", "boxplot", "all"),
  row_col = "row",
  col_col = "col",
  outliers = NULL,
  breaks = "Sturges",
  multiplier = 1.5,
```

```
  scale = c("location", "common"),
  plot_col = intersect(c("plots", "plot_number"), names(data))[1]
)
```

Arguments

`data`	A data frame with one row per plot.
`trait`	Name of the numeric trait column.
`by`	Optional grouping column; by default `"location"` when the data have it.
`type`	`"both"` (histogram and field map), `"histogram"`, `"field"`, `"boxplot"` or `"all"` (boxplot, then histogram and field map).
`row_col, col_col`	Field row and column columns (for the map).
`outliers`	Optional table from `find_outliers()` or `attr(remove_outliers(...), "outliers")`: these plots are marked. By default the outliers are found in `data` with the IQR rule.
`breaks`	Histogram breaks, as in `graphics::hist()`.
`multiplier`	IQR multiplier for the default outliers and the colour scale.
`scale`	`"location"` (default: one colour scale per location, so each field shows its own gradient) or `"common"` (one scale for all locations, to compare their levels).
`plot_col`	Column with the plot numbers used as labels.

Value

Invisibly, a tibble with a summary per group: plots, values, missing, outliers, mean, sd, min, max and CV (%).

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
d <- rc$design
d$yield <- 50 + 0.8 * d$row + rnorm(nrow(d), sd = 2)
d$yield[c(5, 40)] <- c(120, 10)
plot_trait(d, "yield", type = "boxplot")
plot_trait(d, "yield", type = "field")
```

---

prediction_accuracy *Accuracy of adjusted means against known values*

Description

In a simulation the true genotype values are known. This function measures how well an analysis recovered them: the correlation between the adjusted means and the true values.

Usage

```
prediction_accuracy(
  fit,
  truth,
  genotypes = NULL,
  method = c("pearson", "spearman")
)
```

Arguments

fit	An analysis object (from any analyze_*() function) or a data frame with genotype and adjusted_mean columns.
truth	Named numeric vector of true genotype values; names are the genotype names.
genotypes	Optional subset of genotypes to use (for example only the test entries, without the checks).
method	Correlation method: "pearson" (default) or "spearman" (rank correlation, i.e. how well the ranking is recovered).

Value

A single number between -1 and 1.

Examples

```
design <- augmented_prep_design(paste0("G", 1:30), c("A", "B"), block_size = 6, seed = 1)
trial <- design$design
set.seed(1)
truth <- setNames(rnorm(30, sd = 3), paste0("G", 1:30))
trial$yield <- 50 + truth[trial$all_entries] + rnorm(nrow(trial))
fit <- analyze_augmented_prep_design(trial, "yield")
prediction_accuracy(fit, truth)
```

---

prepare_breedbase_pheno

Prepare Breedbase phenotype data for multi-environment analysis

---

Description

This function standardizes Breedbase phenotype data into five columns: environment, genotype, replicate, block, and value.

Usage

```
prepare_breedbase_pheno(
  pheno_raw,
  trait_col,
  location_col = NULL,
  genotype_col = NULL,
  rep_col = NULL,
  block_col = NULL
)
```

Arguments

pheno_raw	A raw Breedbase phenotype data frame, usually read from a Breedbase Excel file.
trait_col	Name of the trait column to analyze.
location_col	Optional location/environment column name. If NULL, the function tries to guess it.
genotype_col	Optional genotype/accession column name. If NULL, the function tries to guess it.
rep_col	Optional replicate column name.
block_col	Optional block column name.

Value

A tibble with standardized phenotype data.

Examples

```
df <- data.frame(
  location = c("Loc1", "Loc1", "Loc2", "Loc2"),
  germplasmName = c("G1", "G2", "G1", "G2"),
  rep = c(1, 1, 1, 1),
  phenotype = c(10, 12, 15, 14)
)

prepare_breedbase_pheno(df, trait_col = "phenotype")
```

---

prepare_breedbase_trial

Prepare an agricolae design and trial information for Breedbase

---

Description

Converts an existing agricolae design and records the information needed to save it as a new Breedbase trial. For example, generate an RCBD (randomized complete block design) with `agricolae::design.rcbd()`, prepare it here, then call `save_breedbase(trial, con)`. Preparation is offline; the randomized allocation, plot numbers and block assignments are kept.

Usage

```
prepare_breedbase_trial(
  design,
  trial_name = NULL,
  year = NULL,
  location = NULL,
  program = NULL,
  description = NULL,
  folder = NULL,
  checks = NULL,
  accession = NULL,
  design_type = NULL,
  trial_type = "phenotyping_trial",
  prompt = interactive(),
  ...
)
```

Arguments

design	An agricolae design, its field-book data frame, or a design already converted with as_breedbase_design().
trial_name	Name of the new trial.
year	Year of the trial, as an integer or integer-valued string.
location	Exact Breedbase location name, for one trial.
program	Exact Breedbase breeding program name.
description	Description of the trial's purpose, material and season.
folder	Optional Breedbase folder name. NULL saves directly in the breeding program. Use create_folder = TRUE in save_breedbase() if a new folder should be created.
checks, accession, design_type	As in as_breedbase_design(). Supply these when converting a raw agricolae design or field book.
trial_type	Breedbase trial type; defaults to "phenotyping_trial".
prompt	Logical. Ask for missing required trial information in the R console. Defaults to TRUE in an interactive session and FALSE in scripts. With FALSE, all missing fields are reported in one error.
...	Additional column mappings for as_breedbase_design(), such as plot, block, rep, row and col.

Details

Required trial information is trial_name, year, location, program and description. Provide it as named arguments for a reproducible script. In RStudio, prepare_breedbase_trial(design) asks for these fields when they are missing.

Inspect trial$trial for the trial information and trial$design for the converted plot table. phenotype_template(trial) uses the recorded trial name to make a field sheet with the plot

names that will be saved. `save_breedbase(trial, dry_run = TRUE)` previews the trial and plot payload without a connection. With a connection, it checks the breeding program, location, accessions and uniqueness of the new trial and plot names. `save_breedbase(trial, con)` then saves through the existing BrAPI workflow. Explicit arguments to the save function override recorded information, for example `trial_name = "Another_trial"`.

No database names or accessions are created during preparation. Physical row and column coordinates are optional for saving; include them in the input field book when known, especially for later spatial analyses. They are not inferred from agricolae plot codes.

Value

A `breedbase_trial` object extending `breedbase_design`, with converted `design`, original book and `parameters`, `design_type` and the recorded `trial` information. Pass the whole object to `save_breedbase()` or `phenotype_template()`.

See Also

`as_breedbase_design()`, `save_breedbase()`, `phenotype_template()`

Examples

```
if (requireNamespace("agricolae", quietly = TRUE)) {
  entries <- c("G1", "G2", "G3", "CHECK_A")
  rcbd <- agricolae::design.rcbd(entries, r = 3, seed = 2026)
  trial <- prepare_breedbase_trial(rcbd,
    trial_name = "RCBD_2026_Ibadan", year = 2026, location = "Ibadan",
    program = "IITA", description = "RCBD with three replicates",
    checks = "CHECK_A", prompt = FALSE)
  trial$trial
  head(trial$design)
  payload <- save_breedbase(trial, dry_run = TRUE)
  sheet <- phenotype_template(trial, traits = "fresh root yield")
}
## Not run:
con <- breedbase_login("https://your-breedbase.org", "your_username")
save_breedbase(trial, con, dry_run = TRUE)  # check names in the database
save_breedbase(trial, con)                  # save the new trial

## End(Not run)
```

---

prepare_design_factors

Prepare design factors

Description

Prepare design factors

Usage

```
prepare_design_factors(data)
```

Arguments

data	Standardized phenotype data.

Value

Data frame with design columns converted to factors.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

str(prepare_design_factors(prepared$data))
```

---

prepare_environment_stratification_data

Prepare phenotype data for Lin environment stratification

Description

Reads a Breedbase-style phenotype data frame and standardizes it for environment stratification.

Usage

```
prepare_environment_stratification_data(pheno, trait, normalize_names = TRUE)
```

Arguments

pheno	A phenotype data frame.
trait	Trait column name.
normalize_names	Logical. If TRUE, normalizes column names.

Value

A list with standardized phenotype data, environment metadata, and selected trait column.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

head(prepared$data)
prepared$env_info
prepared$trait_col
```

---

print.augmented_prep_analysis

Print augmented p-rep analysis

Description

Print augmented p-rep analysis

Usage

```
## S3 method for class 'augmented_prep_analysis'
print(x, ...)
```

Arguments

x	An object returned by analyze_augmented_prep_design().
...	Additional arguments passed to print().

Value

Invisibly returns x.

---

print.augmented_prep_two_stage

Print a two-stage p-rep analysis

Description

Print a two-stage p-rep analysis

Usage

```
## S3 method for class 'augmented_prep_two_stage'
print(x, n = 10, ...)
```

Arguments

x	An object returned by analyze_augmented_prep_two_stage().
n	Number of top-ranked genotypes to show.
...	Unused.

Value

Invisibly returns x.

---

print.augmented_row_column_analysis *Print augmented row-column analysis*

Description

Print augmented row-column analysis

Usage

```
## S3 method for class 'augmented_row_column_analysis'
print(x, ...)
```

Arguments

x	An object returned by analyze_augmented_row_column_design().
...	Additional arguments passed to print().

Value

Invisibly returns x.

---

print.resolvable_row_column_analysis *Print a resolvable row-column analysis*

Description

Print a resolvable row-column analysis

Usage

```
## S3 method for class 'resolvable_row_column_analysis'
print(x, n = 10, ...)
```

Arguments

x	An object returned by analyze_resolvable_row_column().
n	Number of top-ranked treatments to show.
...	Unused.

Value

Invisibly returns x.

---

print.single_stage_analysis

Print a single-stage analysis

Description

Print a single-stage analysis

Usage

```
## S3 method for class 'single_stage_analysis'
print(x, n = 10, ...)
```

Arguments

x	An object returned by analyze_single_stage().
n	Number of top-ranked genotypes to show.
...	Unused.

Value

Invisibly returns x.

---

print.two_stage_analysis

Print a two-stage analysis

Description

Print a two-stage analysis

Usage

```
## S3 method for class 'two_stage_analysis'
print(x, n = 10, ...)
```

Arguments

x	An object returned by analyze_two_stage().
n	Number of top-ranked genotypes to show.
...	Unused.

Value

Invisibly returns x.

---

read_breedbase_file *Read phenotype or design data from Excel, CSV, or tab-delimited files*

Description

Loads phenotype or field-design data from .xlsx, .xls, .csv, .tsv, .txt, or .design files. Excel files are read from all non-empty sheets and combined into one tibble with a .sheet column identifying the source sheet. CSV files are comma-delimited; .tsv, .txt, and .design files are tab-delimited.

Usage

```
read_breedbase_file(file, sheet = NULL, ...)
```

Arguments

file	Path to the phenotype file.
sheet	Optional Excel sheet name or number. If NULL, all non-empty Excel sheets are combined.
...	Additional arguments passed to readxl::read_excel() for Excel files, utils::read.csv() for CSV files, or utils::read.table() for tab-delimited files.

Value

A tibble containing the loaded phenotype data.

Examples

```
# CSV phenotype download
pheno_csv <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
head(pheno_csv)

# Excel workbook: all non-empty sheets are combined
pheno_xlsx <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.xlsx", package = "breedtools")
)
```

```
table(pheno_xlsx$.sheet)

# A single sheet
read_breedbase_file(
  system.file("extdata", "breedbase_pheno.xlsx", package = "breedtools"),
  sheet = 1
)
```

`read_locations` *Load location coordinates*

Description

Loads location metadata from a `.xlsx`, `.xls`, or `.csv` file, or from an already loaded data frame. The returned table has standardized columns: `location`, `latitude`, and `longitude`.

Usage

```
read_locations(
  file,
  location_col = NULL,
  latitude_col = NULL,
  longitude_col = NULL,
  phenotype_data = NULL,
  phenotype_location_col = NULL,
  match_by = "location",
  reference_locations = NULL,
  ignore_case = FALSE,
  sheet = NULL,
  ...
)
```

Arguments

`file`	Path to a location file, or an already loaded data frame.
`location_col`	Optional location name column.
`latitude_col`	Optional latitude column.
`longitude_col`	Optional longitude column.
`phenotype_data`	Optional raw phenotype data frame, prepared phenotype data frame, or result from `run_environment_stratification()`. If supplied, location names are extracted and checked against the location file.
`phenotype_location_col`	Optional location column in `phenotype_data`.
`match_by`	Column to extract when `phenotype_data` is a stratification result. Use `"location"`, `"environment"`, or `"environment_label"`.

reference_locations	Optional character vector of expected location names. If supplied, every reference location must be present in file.
ignore_case	Logical. If TRUE, location matching ignores case.
sheet	Optional Excel sheet name or number passed to read_breedbase_file().
...	Additional arguments passed to read_breedbase_file().

Value

A tibble with location, latitude, and longitude.

Examples

```
locations <- data.frame(
  location = c("Loc1", "Loc2"),
  latitude = c(35.1, 36.2),
  longitude = c(-78.9, -79.4)
)

read_locations(locations)

# Breedbase-style location file, checked against the phenotype locations
loc_file <- system.file("extdata", "breedbase_locations.csv", package = "breedtools")
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
read_locations(loc_file, phenotype_data = pheno)
```

---

remove_outliers *Remove outliers from trait data (IQR rule)*

Description

Finds outliers with find_outliers() and sets those values to NA, so the plots stay in the design but the values are left out of the analysis. The plots and values removed are printed and returned with the data.

Usage

```
remove_outliers(
  data,
  traits,
  multiplier = 1.5,
  by = if ("location" %in% names(data)) "location" else NULL,
  id_cols = c("location", "plot_name", "plots", "plot_number", "all_entries",
    "accession", "row", "col"),
  quiet = FALSE
)
```

Arguments

`data`	A data frame with one row per plot.
`traits`	Names of the numeric trait columns.
`multiplier`	The IQR multiplier k.
`by`	Optional grouping column(s); by default `"location"` when the data have it.
`id_cols`	Columns that identify the plot in the report (those present in `data`).
`quiet`	Logical. If `FALSE` (default), print what was removed.

Value

`data` with the outlying values set to `NA`. The removed values are in `attr(result, "outliers")` (as from `find_outliers()`).

Examples

```
d <- data.frame(plots = 1:20, all_entries = paste0("G", 1:20),
                yield = c(rnorm(18, 50, 2), 80, 10))
clean <- remove_outliers(d, "yield")
attr(clean, "outliers")
```

`resolvable_row_column_design` *Generate a resolvable (spatial) row-column design*

Description

Builds a resolvable row-column design for `v <= k * s` treatments in `r` replicates, each a `k x s` array of rows and columns that contains every treatment once (Williams, John and Whitaker, 2006). The design is found by a search over swaps of treatments within replicates:

1. The average efficiency factor of the resolvable row-column design (rows and columns within replicates), together with the efficiency factors of the component row design and column design, is maximized.
2. With `spatial = TRUE`, the spatial average efficiency factor of the two-dimensional linear variance (LV + RC) model is then maximized while keeping the three efficiency factors of step 1 within `efficiency_tolerance` of their optimized values. This spreads treatments over edge and interior plots and over row and column neighbours, without needing prior values for spatial parameters.

When replicates are contiguous, `latinize = TRUE` builds a latinized design: no treatment appears twice in the same long column (replicates stacked vertically) or long row (replicates side by side), within every group of `latin_t` adjacent replicates (t-latinized designs; John and Williams, 1998).

Usage

```
resolvable_row_column_design(
  treatments,
  n_reps,
  rows_per_rep,
  cols_per_rep,
  spatial = TRUE,
  latinize = FALSE,
  replicate_layout = c("vertical", "horizontal"),
  latin_t = NULL,
  n_starts = 3,
  n_iter = NULL,
  efficiency_tolerance = 0.002,
  plot_type = c("serpentine", "cartesian"),
  seed = NULL,
  output_file = NULL,
  start_design = NULL,
  trial_name = NULL,
  location = NULL,
  plot_name_pattern = "{trial}_{location}_{accession}_{block}_{rep}"
)
```

Arguments

`treatments`	Character vector of treatment names (`v <= k * s`).
`n_reps`	Number of replicates (`r`).
`rows_per_rep`	Number of rows in each replicate (`k`).
`cols_per_rep`	Number of columns in each replicate (`s`).
`spatial`	Logical. If `TRUE` (default), optimize the spatial efficiency factor in a second stage.
`latinize`	Logical. If `TRUE`, build a latinized design for contiguous replicates.
`replicate_layout`	How replicates are placed in the field: `"vertical"` (stacked, giving long columns) or `"horizontal"` (side by side, giving long rows).
`latin_t`	Optional size of the groups of adjacent replicates within which latinization holds (t-latinized design). Defaults to all replicates.
`n_starts`	Number of random starting designs.
`n_iter`	Number of swap proposals per stage and start. Defaults to `min(20000, max(3000, 60 * v * r))`.
`efficiency_tolerance`	Largest loss in the row-column, row and column efficiency factors accepted in the spatial stage.
`plot_type`	Plot numbering order: `"serpentine"` or `"cartesian"`.
`seed`	Optional random seed.
`output_file`	Optional tab-delimited output file for the design.

start_design — Optional existing design to start from (for example one produced by CycDesigN), as a data frame with replicate, row_in_rep, col_in_rep and all_entries columns and complete replicates. The search then improves this design; use n_iter = 1 and spatial = FALSE to keep it essentially as it is and only lay it out.

trial_name — Optional Breedbase trial name. When given, a plot_name column with the plot names used by save_breedbase() is added to the design (see add_plot_names()), so the names can be checked with breedbase_check() and changed before saving.

location — Breedbase location of the trial, for the {location} field of the plot names. Used only with trial_name.

plot_name_pattern — Pattern for the plot names, as in add_plot_names(). Used only with trial_name.

Details

When there are fewer treatments than plots per replicate, the remaining plots are fillers (all_entries = "FILLER", is_filler = 1), placed by the search like an extra treatment.

Value

An object of class resolvable_row_column_design: a list with input, summary (efficiency factors, spatial upper bound, latinization check), design (one row per plot with replicate, row_in_rep, col_in_rep, field row and col, long_row / long_col, all_entries, is_filler), breedbase_design, and efficiency (see resolvable_row_column_efficiency()).

References

Williams, E. R., John, J. A. and Whitaker, D. (2006). Construction of resolvable spatial row-column designs. Biometrics 62, 103-108. doi:10.1111/j.15410420.2005.00393.x

John, J. A. and Williams, E. R. (1998). t-Latinized designs. Australian and New Zealand Journal of Statistics 40, 111-118.

See Also

resolvable_row_column_efficiency(), analyze_resolvable_row_column(), plot_resolvable_row_column_desig

Examples

```
# Example 2 of Williams, John and Whitaker (2006): 42 treatments in three
# replicates of 6 rows x 7 columns.
design <- resolvable_row_column_design(
  treatments = paste0("T", 1:42),
  n_reps = 3,
  rows_per_rep = 6,
  cols_per_rep = 7,
  n_starts = 1,
  n_iter = 2000,
  seed = 1
```

```
)
design$summary
```

`resolvable_row_column_efficiency`
Efficiency of a resolvable row-column design

Description

Computes the design criteria of Williams, John and Whitaker (2006) for a resolvable row-column design with complete replicates: the average efficiency factors of the row-column design, the row design and the column design (rows and columns nested within replicates), the spatial average efficiency factor of the two-dimensional linear variance model,

$$A = rI - X'(I_r \otimes W_R \otimes W_C)X,$$

with its upper bound $U = (v - t)/(v - 1)$, $t = tr(W_R)tr(W_C)$, and treatment neighbour and edge statistics. Optionally, it evaluates the average pairwise variance criterion $z(X)$ under an assumed autoregressive (AR) model, which is used to compare designs (Section 4 of the paper).

Usage

```
resolvable_row_column_efficiency(
  design,
  ar = NULL,
  genotype_col = "all_entries",
  replicate_col = "replicate",
  row_col = "row_in_rep",
  col_col = "col_in_rep"
)
```

Arguments

`design`	A design object from `resolvable_row_column_design()` or a data frame with replicate, row, column and treatment columns. Each replicate must be complete (`k * s` plots, each treatment once; filler plots count as separate treatments).
`ar`	Optional named list of AR model parameters `eta_r`, `eta_c`, `psi`, `rho_r`, `rho_c` (Eq. 4 of the paper).
`genotype_col, replicate_col, row_col, col_col`	Column names.

Value

A list with `efficiency_rc`, `efficiency_rows`, `efficiency_cols`, `efficiency_spatial`, `upper_bound_spatial`, `ar_criterion` (`NA` unless `ar` is given), and `neighbours`, a tibble with each treatment's number of distinct row and column neighbours and the number of replicates in which it is on an edge or in a corner.

Examples

```
design <- resolvable_row_column_design(paste0("T", 1:20), 2, 4, 5,
                                       n_starts = 1, n_iter = 500, seed = 1)
eff <- resolvable_row_column_efficiency(design,
  ar = list(eta_r = 35, eta_c = 60, psi = 10, rho_r = 0.5, rho_c = 0.5))
eff[c("efficiency_rc", "efficiency_spatial", "ar_criterion")]
```

`run_breedbase_environment_stratification` *Run Lin-style environment stratification from a Breedbase phenotype file*

Description

Reads a Breedbase phenotype file, prepares the phenotype data, detects the experimental design, calculates ANOVA, and groups environments using a Lin-style genotype-by-environment interaction test.

Usage

```
run_breedbase_environment_stratification(
  file,
  trait_col,
  alpha = 0.05,
  normalize_names = TRUE
)
```

Arguments

`file`	Path to a `.xlsx`, `.xls`, or `.csv` phenotype file, or an already loaded data frame.
`trait_col`	Trait column name.
`alpha`	Significance level. Default is `0.05`.
`normalize_names`	Logical. If `TRUE`, normalize Breedbase-style column names.

Value

A list containing standardized phenotype data, environment metadata, ANOVA results, pairwise environment tests, group summary, group membership, ungrouped environments, summary, and message.

Examples

```
pheno_file <- system.file("extdata", "breedbase_pheno.csv", package = "breedtools")

result <- run_breedbase_environment_stratification(
  file = pheno_file,
  trait_col = "fresh root yield",
  alpha = 0.05
)

result$group_summary
result$group_membership
```

---

run_breedbase_variance_check

Run Breedbase multi-location phenotype variance check

Description

Reads a Breedbase phenotype file, runs individual ANOVA by environment, performs Hartley Fmax tests, and identifies compatible environment groups for joint analysis.

Usage

```
run_breedbase_variance_check(
  file,
  trait_col,
  alpha = 0.05,
  location_col = NULL,
  genotype_col = NULL,
  rep_col = NULL,
  block_col = NULL,
  max_group_size = Inf,
  n_sim = 1e+05,
  seed = 123,
  export = FALSE,
  output_dir = NULL
)
```

Arguments

`file`	Path to a `.xlsx`, `.xls`, or `.csv` phenotype file, or an already loaded data frame.
`trait_col`	Trait column to analyze.
`alpha`	Significance level for Hartley Fmax test.
`location_col`	Optional location/environment column.
`genotype_col`	Optional genotype/accession column.

rep_col	Optional replicate column.
block_col	Optional block column.
max_group_size	Maximum environment group size to test.
n_sim	Number of simulations for Hartley critical value.
seed	Random seed for the Hartley critical value simulation. The user's global random number generator state is not modified.
export	Logical. If TRUE, writes CSV files to output_dir.
output_dir	Directory where CSV files should be written. Defaults to NULL, in which case nothing is written; it must be supplied when export = TRUE.

Value

A list containing cleaned data, ANOVA results, Hartley test, all group tests, compatible groups, maximal compatible groups, and detected columns.

Examples

```
pheno_file <- system.file("extdata", "breedbase_pheno.csv", package = "breedtools")

result <- run_breedbase_variance_check(
  file = pheno_file,
  trait_col = "fresh root yield|CO_334:0000013",
  n_sim = 2000
)

result$env_anova
result$maximal_compatible_groups

# Export CSV files to a temporary directory
out_dir <- file.path(tempdir(), "variance_check")
result <- run_breedbase_variance_check(
  file = pheno_file,
  trait_col = "fresh root yield|CO_334:0000013",
  n_sim = 2000,
  export = TRUE,
  output_dir = out_dir
)
list.files(out_dir)
unlink(out_dir, recursive = TRUE)
```

---

run_environment_stratification

Run Lin-style environment stratification

Description

This function reads phenotype data, detects design structure, calculates ANOVA, tests genotype-by-environment interaction for environment groups, and returns compatible environment groups.

Usage

```
run_environment_stratification(
  phenotype_file,
  trait,
  alpha = 0.05,
  sep = NULL,
  normalize_names = TRUE,
  export_json = FALSE,
  output_dir = NULL,
  prefix = "environment_stratification"
)
```

Arguments

phenotype_file	Path to a .xlsx, .xls, or .csv phenotype file, or an already loaded data frame.
trait	Trait column name.
alpha	Significance level. Default is 0.05.
sep	Deprecated. File delimiters are detected from the extension by read_breedbase_file().
normalize_names	Logical. If TRUE, normalize Breedbase column names.
export_json	Logical. If TRUE, writes a JSON result file to output_dir.
output_dir	Output directory for the JSON file. Defaults to NULL, in which case nothing is written; it must be supplied when export_json = TRUE.
prefix	Prefix for JSON output files.

Value

A list with summary, ANOVA results, pairwise tests, group summary, group membership, ungrouped environments, and message.

Examples

```
pheno_file <- system.file("extdata", "breedbase_pheno.csv", package = "breedtools")

result <- run_environment_stratification(
  phenotype_file = pheno_file,
  trait = "fresh root yield",
  alpha = 0.05
)

result$group_summary
result$group_membership
```

```
# Export the result as JSON to a temporary directory
out_dir <- file.path(tempdir(), "stratification")
result <- run_environment_stratification(
  phenotype_file = pheno_file,
  trait = "fresh root yield",
  export_json = TRUE,
  output_dir = out_dir
)
list.files(out_dir)
unlink(out_dir, recursive = TRUE)
```

---

save_breedbase *Save a design to Breedbase*

Description

Creates the trial and its plots in a Breedbase database through BrAPI, in a breeding program and, optionally, a folder of your choice. Works for designs from `augmented_row_column_design()`, `augmented_prep_design()`, `resolvable_row_column_design()`, and agricolae designs converted with `as_breedbase_design()` or prepared with `prepare_breedbase_trial()`.

Usage

```
save_breedbase(
  design,
  con = NULL,
  program = NULL,
  folder = NULL,
  trial_name,
  year,
  location = NULL,
  description,
  trial_type = "phenotyping_trial",
  design_type = NULL,
  plot_names = NULL,
  plot_name_pattern = NULL,
  create_folder = FALSE,
  dry_run = FALSE,
  confirm = interactive()
)
```

Arguments

design	A design object, including the result of `as_breedbase_design()` or `prepare_breedbase_trial()`. For a prepared trial, recorded trial name, year, location, program, description, folder and trial type are used when their arguments are omitted. Explicit arguments override them.

con	A connection from breedbase_login().
program	Breeding program name.
folder	Optional folder inside the program. Without a folder (the default, NULL) the trial is saved directly in the breeding program. An existing folder is used as it is; a new one is created when create_folder = TRUE.
trial_name	Name of the new trial (must not exist yet). In the names saved in Breedbase (trial, plot and new folder names, and the location part of p-rep trial names) spaces and special characters such as % $ ( ) { } [ ] \\ / < > * are replaced with "_" (only letters, digits, _, - and . are kept), e.g. "Alcobaca (BA)" gives "Alcobaca_BA". Accession and location names are kept as they are, to match Breedbase. [ ]: R:%20
year	Year of the trial.
location	Breedbase location name. For p-rep designs, a named vector mapping the design's locations to Breedbase locations, e.g. c(North = "Ibadan", South = "Ubiaja"); by default the design's location names are used.
description	Description of the trial (required), e.g. its purpose, material and season: "Advanced yield trial of 108 lines and 4 checks, Ibadan 2026". It is shown on the trial page in Breedbase.
trial_type	Breedbase trial type, e.g. "Preliminary Yield Trial". Types the database does not know are saved as a miscellaneous trial.
design_type	Breedbase design type. Defaults to "Augmented" for augmented row-column designs, "p-rep" for p-rep designs and "Alpha" (rows as incomplete blocks within replicates) for resolvable row-column designs. Converted agricolae designs use the type selected by as_breedbase_design().
plot_names	Optional plot names given by you: either a character vector with one name per plot, in the row order of design$design, or the name of a column of design$design that holds them. Names must be unique.
plot_name_pattern	Pattern for automatic plot names, used when plot_names is NULL. With NULL (the default) the plot_name column of the design is used when it has one, and otherwise the pattern "{trial}_{location}_{accession}_{block}_{rep}", which gives names like "AYT_2026_Ibadan_LINE_007_3_1". Available fields: {trial} (the Breedbase trial name), {location} (the Breedbase location, with spaces and special characters replaced by "_"), {accession}, {block}, {rep}, {plot} (plot number), {row} and {col}. If a pattern gives the same name to two plots (the same accession twice in one block and rep), the plot number is added to those names.
create_folder	Logical. Create folder if it does not exist.
dry_run	Logical. Make all checks but write nothing.
confirm	Logical. Ask before writing (default in interactive sessions).

Details

Before anything is written, every name is checked: the breeding program, the folder if given (created when create_folder = TRUE), the location(s), that every entry and check already exists as an

accession, and that the trial name is free. In an interactive session you are then asked to confirm. After saving, the plots are read back from Breedbase to verify the upload.

Each plot is saved with its name, plot number, row and column, rep, block and whether it is a check, and its GPS coordinates when the design has them (see `add_plot_coordinates()`; they are sent just after the plots and read back to check). Plot names stored in the design (a `plot_name` column, made by the design functions when given a `trial_name` or by `add_plot_names()`) are used as they are, so they can be reviewed and changed before saving. Otherwise they are made automatically from the trial name, accession, block and rep (see `plot_name_pattern`), or you can give your own with `plot_names`. Plot names must be unique in the whole database, which is checked before saving. A multi-location p-rep design becomes one Breedbase trial per location, named `"<trial_name>_<location>"`. Filler plots of resolvable row-column designs are not saved.

Run first with `dry_run = TRUE`: all the checks are made, but nothing is written. Without a connection, `dry_run = TRUE` returns the data that would be sent, without contacting any server.

Value

Invisibly, a tibble with one row per Breedbase trial: location, trial name, trial id, number of plots sent and found after saving, number of plots with GPS coordinates sent and found, and the address of the trial page. With `dry_run = TRUE` and no connection, a list with the trial and plot data that would be sent.

Slow servers

Accessions are searched 25 names at a time, because Breedbase looks up the pedigree of each one. If the server times out (HTTP 504, 502, 503 or 429), the batch is split in half and tried again, so a busy server is slower but still works. A line is printed after each batch with the names done and the time. Accessions found are remembered until R is restarted, so `save_breedbase()` after `breedbase_check()` does not search them again.

When creating the folder, the trial or its plots times out, the save does not stop: the server may still finish the job, so it checks for the result every 20 seconds for 3 minutes, and tries once more if it is not there (Breedbase refuses names that already exist, so this cannot make duplicates). If a save is cut off anyway, run `save_breedbase()` again: an existing folder is used, and a trial left without plots gets its plots. Settings: `options(breedtools.write_wait = 30, breedtools.write_checks = 10)`. Settings: `options(breedtools.germplasm_batch = 10)` for smaller batches, `options(breedtools.breedbase_timeout = 600)` to wait longer (seconds) for each answer.

See Also

`breedbase_login()`, `breedbase_programs()`

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
# What would be sent (no server needed):
payload <- save_breedbase(rc, trial_name = "AYT_2026_Ithaca", year = 2026,
                          location = "Ithaca",
                         description = "Advanced yield trial of 108 lines and 4 checks",
```

```
                            dry_run = TRUE)
str(payload$studies[[1]], max.level = 1)
payload$plots[[1]][[1]]

## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
save_breedbase(rc, con, program = "IITA", folder = "AYT 2026",
               trial_name = "AYT_2026_Ibadan", year = 2026, location = "Ibadan",
             description = "Advanced yield trial of 108 lines and 4 checks, Ibadan 2026",
               create_folder = TRUE, dry_run = TRUE)   # check first
save_breedbase(rc, con, program = "IITA", folder = "AYT 2026",
               trial_name = "AYT_2026_Ibadan", year = 2026, location = "Ibadan",
             description = "Advanced yield trial of 108 lines and 4 checks, Ibadan 2026")

## End(Not run)
```

`save_breedbase_analysis` *Save analysis results to Breedbase as an analysis*

Description

Saves analysis results as a Breedbase analysis (a project of type analysis), through the same upload as the "Upload analysis" form of Breedbase:

`level = "accession"` "Analysis of: accessions": one value per genotype, e.g. second-stage adjusted means from `second_stage_values()`.

`level = "plot"` "Analysis of: plots": one value per field plot, e.g. first-stage values from `first_stage_values()`. Breedbase finds the field trial(s) of the plots and gives the analysis one plot per field plot (named "<analysis_name>_<field plot name>"), with the same accession, rep, block, row and column.

Each trait is saved together with a statistic term, as Breedbase does for every analysis. The analysis then appears on the Analyses page and can be used in later analyses. Every name (analysis, accessions or plots, traits, statistic) is checked first, and all problems are listed before anything is written. The field trials do not change.

Usage

```
save_breedbase_analysis(
  data,
  con = NULL,
  analysis_name,
  program,
  description,
  year = format(Sys.Date(), "%Y"),
  level = c("accession", "plot"),
```

```
  traits = NULL,
  accession = "accession",
  plot = "plot_name",
  estimate = NULL,
  statistic = NULL,
  protocol = NULL,
  time_terms = NULL,
  upload_file = NULL,
  dry_run = FALSE,
  confirm = interactive(),
  folder = NULL,
  create_folder = FALSE
)
```

Arguments

data	A data frame with one row per accession (level = "accession") or per plot (level = "plot") and one numeric column per trait.
con	A connection from breedbase_login(); NULL with dry_run = TRUE returns the file that would be uploaded.
analysis_name	Name of the new analysis (must not exist yet). Spaces and special characters are replaced with "_" (only letters, digits, _, - and . are kept).
program	Breeding program of the analysis.
description	Description of the analysis (required).
year	Year of the analysis (default: this year).
level	"accession" or "plot" (Breedbase's "Analysis of").
traits	Trait columns (default: all except the name and design columns). Their names are the Breedbase trait names.
accession, plot	Names of the accession and plot-name columns.
estimate	For the default statistic: "blue" or "blup" (adjusted means of that kind), or "spatial" (spatial correction, the default for level = "plot"). By default taken from the data (second_stage_values(), first_stage_values()).
statistic	Optional statistic term: its name, "name|SGNSTAT:id" or the id, as listed by breedbase_statistics(). It must be a term of the Breedbase statistics ontology (checked against the ontology file, with suggestions for misspelled names, and then on the server). By default chosen by estimate.
protocol	The model, e.g. a formula: "SpATS(yield ~ genotype + PSANOVA(col, row))". Default: the description.
time_terms	Optional time terms per trait, as in the form's "Compose Trait Info With Time Term", e.g. c("fresh root yield|CO_334:0000013" = "day 300|TIME:0000300").
upload_file	Where to write the upload file when the site's web pages are closed by a web password (see Details). Default: the analysis name + "_upload.csv" in the working folder.
dry_run	Logical. Check everything but write nothing.

confirm	Logical. Ask before writing (default in interactive sessions).
folder	Optional folder of the program to put the analysis in, e.g. the folder of its trials. Without a folder (the default, NULL) the analysis is in the breeding program itself. An existing folder is used as it is; a new one (special characters replaced with "_") is created when create_folder = TRUE.
create_folder	Logical. Create folder when it does not exist.

Details

The form's fields map to the arguments: Analysis Name (analysis_name, unique), Analysis Description (description), Analysis Protocol (protocol), Analysis Of (level), Breeding Program (program), Statistical Ontology Term (statistic) and the optional time terms (time_terms). The model type is Breedbase's own one for uploaded results (uploaded_generic_analysis_model).

Value

Invisibly, a tibble with the analysis name and id, the number of rows (accessions or plots) sent and read back, and the analysis page; or, when the web pages are closed, a list with the upload file and the form values. A dry run returns the file, the form and traits: each trait with the composed name it will be stored under (stored_as) and status ("exists" or "new").

Sites with a web password

Breedbase creates analyses through its web pages; BrAPI cannot create an analysis project. Test sites such as cassava-test.sgn.cornell.edu close their web pages with an extra web password. With it (see breedbase_login()), the analysis is uploaded directly. Without it, everything is checked through BrAPI and the file of the "Upload analysis" form is written, with the value of every form field printed: upload it in the browser and Breedbase creates the same analysis project.

The stored trait names

Breedbase stores the values of an analysis under the trait composed with the statistic, "trait|statistic|COMP:id", e.g. "fresh root yield|Adjusted Means from Spatial Correction using SpATS R|COMP:0000379", never under the plain trait. Before saving (and in a dry run) each trait is listed with the composed name it will get, and whether that composed trait already exists or Breedbase will create it. To use another statistic, list them with breedbase_statistics() and give one with statistic; run with dry_run = TRUE to see the names again.

Examples

```
prep <- augmented_prep_design(paste0("E", 1:60), locations = c("North", "South"), seed = 1)
d <- prep$design
set.seed(1)
d$yield <- rnorm(nrow(d), 25, 3)
fit <- analyze_augmented_prep_two_stage(d, "yield")
means <- second_stage_values(fit, trait = "fresh root yield")
sent <- save_breedbase_analysis(means, analysis_name = "PREP_2026_stage2", program = "IITA",
                                description = "Two-stage analysis, weighted", dry_run = TRUE)
head(sent$file)
```

```
## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
# Second stage: one value per accession.
save_breedbase_analysis(means, con, analysis_name = "PREP_2026_stage2", program = "IITA",
                        description = "Two-stage analysis, weighted")
# First stage: one value per field plot ("Analysis of: plots").
stage1 <- first_stage_values(fit, prep, "PREP_2026", trait = "fresh root yield")
save_breedbase_analysis(stage1, con, analysis_name = "PREP_2026_stage1", program = "IITA",
                        description = "SpATS per location", level = "plot")

## End(Not run)
```

`save_breedbase_coordinates` *Save plot GPS coordinates of a trial already in Breedbase*

Description

Sends the plot coordinates of a design (see `add_plot_coordinates()`) to a trial that is already saved in Breedbase, e.g. when the plots were measured after the trial was saved. Plots are matched by plot number, after checking that every plot has the same accession in Breedbase as in the design. Coordinates already stored for these plots are replaced. `save_breedbase()` does this by itself for a design that has coordinates.

Usage

```
save_breedbase_coordinates(
  design,
  con,
  trial_name = NULL,
  dry_run = FALSE,
  confirm = interactive()
)
```

Arguments

`design`	A design with coordinates, from `add_plot_coordinates()`.
`con`	A connection from `breedbase_login()`.
`trial_name`	The trial name in Breedbase. For a multi-location p-rep design, the location is added as in `save_breedbase()`. Can be omitted for a prepared Breedbase trial or a design with plot names.
`dry_run`	Logical. Check the trial and plots but write nothing.
`confirm`	Logical. Ask before writing (default in interactive sessions).

Value

Invisibly, a tibble with one row per trial: `trial_name`, `plots_with_gps` (sent) and `gps_found` (read back from Breedbase).

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
rc <- add_plot_coordinates(rc, origin = c(7.4931, 3.9035), plot_width = 3, plot_length = 5)
save_breedbase_coordinates(rc, con, trial_name = "AYT_2026_Ibadan")

## End(Not run)
```

`save_breedbase_first_stage` *Save first-stage results to Breedbase as a trait of the field plots*

Description

Saves plot-level first-stage results (from `first_stage_values()`) on the plots of the field trial, as values of a trait made for adjusted values, so raw and adjusted values stay apart. By default that trait is the measured trait composed with a statistic, e.g. "fresh root yield|Adjusted Means from Spatial Correction using SpATS R": it is found, or created (after you confirm) when it does not exist yet; see `breedbase_adjusted_trait()`. Or name your own traits with `as_trait`. The values are saved through BrAPI, like `save_breedbase_phenotypes()`.

Usage

```
save_breedbase_first_stage(
  values,
  con = NULL,
  as_trait = NULL,
  statistic = NULL,
  create_trait = TRUE,
  dry_run = FALSE,
  confirm = interactive(),
  overwrite = FALSE,
  remove_empty = FALSE
)
```

Arguments

`values`	A table from `first_stage_values()` (or any table with a `plot_name` column and one numeric column per trait).
`con`	A connection from `breedbase_login()`; `NULL` with `dry_run = TRUE` returns what would be sent.

`as_trait` Optional named vector: for each trait column of `values`, an existing Breedbase trait to save it under, e.g. `c("fresh root yield" = "fresh root yield adjusted|COMP:0000123")`.

`statistic` The statistic composed with each trait when `as_trait` is not given. By default chosen from this Breedbase's own statistics (the spatial correction term); see `breedbase_statistics()`.

`create_trait` Logical. Create the composed traits that do not exist yet (you are asked first when `confirm = TRUE`).

`dry_run`, `confirm`, `overwrite`, `remove_empty`
As in `save_breedbase_phenotypes()`: run it again after a new analysis with `overwrite = TRUE` to replace the stored values.

Value

As `save_breedbase_phenotypes()`: one row per trait with the values sent and found.

Slow servers

Accessions are searched 25 names at a time, because Breedbase looks up the pedigree of each one. If the server times out (HTTP 504, 502, 503 or 429), the batch is split in half and tried again, so a busy server is slower but still works. A line is printed after each batch with the names done and the time. Accessions found are remembered until R is restarted, so `save_breedbase()` after `breedbase_check()` does not search them again.

When creating the folder, the trial or its plots times out, the save does not stop: the server may still finish the job, so it checks for the result every 20 seconds for 3 minutes, and tries once more if it is not there (Breedbase refuses names that already exist, so this cannot make duplicates). If a save is cut off anyway, run `save_breedbase()` again: an existing folder is used, and a trial left without plots gets its plots. Settings: `options(breedtools.write_wait = 30, breedtools.write_checks = 10)`. Settings: `options(breedtools.germplasm_batch = 10)` for smaller batches, `options(breedtools.breedbase_timeout = 600)` to wait longer (seconds) for each answer.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
d <- rc$design
set.seed(1)
d$yield <- 25 + 0.4 * d$row + rnorm(nrow(d), sd = 2)
fit <- analyze_augmented_row_column_design(d, "yield")
values <- first_stage_values(fit, rc, "AYT_2026_Ibadan", trait = "fresh root yield",
                             location = "Ibadan")
obs <- save_breedbase_first_stage(values, dry_run = TRUE)
head(obs)

## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
save_breedbase_first_stage(values, con, dry_run = TRUE)  # check; shows the trait to create
save_breedbase_first_stage(values, con)              # creates the trait if needed, saves
```

```
## End(Not run)
```

`save_breedbase_phenotypes` *Save phenotypic data to Breedbase*

Description

Uploads trait values of plots that are already in Breedbase (for example saved with `save_breedbase()`) through BrAPI. Give the plot name and the trait name with each value; the plots and traits are looked up in Breedbase first, and every problem is listed before anything is written.

Usage

```
save_breedbase_phenotypes(
  data,
  con = NULL,
  plot = "plot_name",
  traits = NULL,
  trait = NULL,
  value = NULL,
  timestamp = NULL,
  overwrite = FALSE,
  remove_empty = FALSE,
  dry_run = FALSE,
  confirm = interactive(),
  refresh = TRUE
)
```

Arguments

`data`	A data frame with the plot names and the values.
`con`	A connection from `breedbase_login()`; `NULL` with `dry_run = TRUE` returns the observations without checking them.
`plot`	Name of the column with the plot names.
`traits`	For wide data: the trait columns. By default every column except `plot`, `timestamp` and the design columns of `phenotype_template()` (`trial_name`, `accession`, `plot_number`, `block`, `rep`, `row`, `col`, `is_check`).
`trait, value`	For long data: the names of the trait and value columns.
`timestamp`	Optional name of a column with the date and time of each measurement (e.g. `"2026-08-15 10:30:00"`). Breedbase uses it to tell repeated measurements apart.
`overwrite`	Logical. Replace values that Breedbase already has with different ones (see "Saving again").

`remove_empty` Logical. Delete stored values whose cell is empty (`NA` or `""`) in `data`; needs `overwrite = TRUE` (see "Saving again").

`dry_run` Logical. Check everything but write nothing.

`confirm` Logical. Ask before writing (default in interactive sessions).

`refresh` Logical. After saving, refresh the phenotype cache of Breedbase so the values show on the trial pages (see `breedbase_refresh()`).

Details

The data can be wide, one row per plot and one column per trait (as made by `phenotype_template()`):

plot_name	fresh root yield	dry matter content percentage
AYT_2026_Ibadan_G31_1_1	25.4	31.2

or long, one row per value, with `trait` and `value` columns (`trait = "trait", value = "value"`).

Trait names are the Breedbase names, with or without the ontology id: `"fresh root yield"` or `"fresh root yield|CO_334:0000013"` (the column names of a Breedbase phenotype download). Values are checked by Breedbase against the trait (e.g. numbers within the allowed range).

Value

Invisibly, a tibble with one row per trait: `trait`, `observation_variable_db_id`, `values_sent`, `values_found` (read back from Breedbase after saving), and how the values were handled: `new`, `replaced`, `unchanged` and `removed`. Returns invisible `NULL` when saving is declined, including changed values with `overwrite = FALSE`.

Saving again (values already in Breedbase)

Before writing, the values already stored for these plots and traits are read, and every value is sorted into new, unchanged (same value, not sent again) or changed:

`overwrite = FALSE` **(default)** If changed values exist, a message is printed and the function returns without an error; nothing is written.

`overwrite = TRUE` Changed values replace the stored ones.

Empty cells (`NA` or `""`) are never sent. When Breedbase has a value for an empty cell (e.g. an outlier set to `NA` with `remove_outliers()`):

`remove_empty = FALSE` **(default)** The stored value is kept (and reported).

`remove_empty = TRUE` The stored value is deleted. Like the "remove values" option of Breedbase's upload page, this needs `overwrite = TRUE`, a curator account and access to the web pages (the site password, see `breedbase_login()`).

Slow servers

Accessions are searched 25 names at a time, because Breedbase looks up the pedigree of each one. If the server times out (HTTP 504, 502, 503 or 429), the batch is split in half and tried again, so a busy server is slower but still works. A line is printed after each batch with the names done and the time. Accessions found are remembered until R is restarted, so `save_breedbase()` after `breedbase_check()` does not search them again.

When creating the folder, the trial or its plots times out, the save does not stop: the server may still finish the job, so it checks for the result every 20 seconds for 3 minutes, and tries once more if it is not there (Breedbase refuses names that already exist, so this cannot make duplicates). If a save is cut off anyway, run `save_breedbase()` again: an existing folder is used, and a trial left without plots gets its plots. Settings: `options(breedtools.write_wait = 30, breedtools.write_checks = 10)`. Settings: `options(breedtools.germplasm_batch = 10)` for smaller batches, `options(breedtools.breedbase_timeout = 600)` to wait longer (seconds) for each answer.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
sheet <- phenotype_template(rc, "AYT_2026", location = "Ibadan", traits = "fresh root yield")
sheet$`fresh root yield` <- round(rnorm(nrow(sheet), 25, 4), 1)

# What would be sent (no connection needed):
obs <- save_breedbase_phenotypes(sheet, dry_run = TRUE)
head(obs)

## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
save_breedbase_phenotypes(sheet, con, dry_run = TRUE)   # check first
save_breedbase_phenotypes(sheet, con)

# Saving a corrected sheet again: replace changed values, and delete the
# stored values of cells now empty (e.g. outliers set to NA).
save_breedbase_phenotypes(sheet, con, overwrite = TRUE, remove_empty = TRUE)

# Long data: one row per value
long <- data.frame(plot_name = sheet$plot_name, trait = "fresh root yield",
                   value = sheet$`fresh root yield`)
save_breedbase_phenotypes(long, con, trait = "trait", value = "value")

## End(Not run)
```

---

second_stage_values *Second-stage genotype values from analyses*

Description

Collects the adjusted means of analyses (one per trait) into one table with one row per accession, ready for save_breedbase_analysis().

Usage

```
second_stage_values(fits, trait = NULL)
```

Arguments

fits	One analysis, or a named list of analyses, one per trait (the names are the Breedbase trait names). Any breedtools analysis with adjusted_means, e.g. analyze_augmented_prep_two_stage().
trait	Trait name when fits is a single analysis.

Value

A tibble with accession and one column per trait. Its attribute "estimate" is "blup" when the genotypes were random, else "blue".

Examples

```
prep <- augmented_prep_design(paste0("E", 1:60), locations = c("North", "South"), seed = 1)
d <- prep$design
set.seed(1)
d$yield <- rnorm(nrow(d), 25, 3)
fit <- analyze_augmented_prep_two_stage(d, "yield")
head(second_stage_values(fit, trait = "fresh root yield"))
```

Index

add_breedbase_analysis_values, 7
add_breedbase_analysis_values(), *13*, *44*
add_environment_metadata, 8
add_pairwise_environment_metadata, 9
add_plot_coordinates, 10
add_plot_coordinates(), *102*, *106*
add_plot_names, 12
add_plot_names(), *34*, *40*, *45*, *74*, *94*, *102*
analysis_plot_names, 13
analyze_augmented_prep_design, 14
analyze_augmented_prep_design(), *5*, *17–19*, *29*, *35*, *64*
analyze_augmented_prep_two_stage, 17
analyze_augmented_prep_two_stage(), *5*, *16*, *27*, *29*, *57*, *64*, *112*
analyze_augmented_row_column_design, 20
analyze_augmented_row_column_design(), *5*, *29*, *57*, *64*
analyze_environment_anova, 22
analyze_environment_anova(), *72*
analyze_one_environment, 23
analyze_resolvable_row_column, 24
analyze_resolvable_row_column(), *5*, *29*, *64*, *88*, *94*
analyze_single_stage, 26
analyze_single_stage(), *6*, *48*, *88*
analyze_two_stage, 27
analyze_two_stage(), *6*, *26*, *48*, *57*, *89*
as_breedbase_design, 30
as_breedbase_design(), *6*, *10*, *12*, *45*, *74*, *83, 84*, *100, 101*
augmented_prep_concurrence, 31
augmented_prep_concurrence(), *5*
augmented_prep_design, 32
augmented_prep_design(), *5*, *10*, *12*, *17, 18*, *45*, *52*, *61*, *74*, *100*
augmented_prep_parameters, 36
augmented_prep_parameters(), *5*, *35*, *61*
augmented_row_column_capacity, 37
augmented_row_column_capacity(), *5*, *39–42*
augmented_row_column_design, 38
augmented_row_column_design(), *5*, *10*, *12*, *42*, *45*, *52*, *60, 61*, *74*, *100*
augmented_row_column_layouts, 41
augmented_row_column_layouts(), *5*, *61*

breedbase_adjusted_trait, 43
breedbase_adjusted_trait(), *6*, *8*, *107*
breedbase_analysis_plots, 44
breedbase_analysis_plots(), *13*
breedbase_check, 45
breedbase_check(), *6*, *12*, *34*, *40*, *46*, *94*, *102*, *108*, *111*
breedbase_folders *(*breedbase_lists*)*, 46
breedbase_folders(), *6*, *48*
breedbase_lists, 46
breedbase_locations *(*breedbase_lists*)*, 46
breedbase_locations(), *6*, *48*
breedbase_login, 47
breedbase_login(), *6, 7*, *43–45*, *47–51*, *101, 102*, *104–107*, *109, 110*
breedbase_phenotypes, 48
breedbase_phenotypes(), *6*, *26*, *64*
breedbase_programs *(*breedbase_lists*)*, 46
breedbase_programs(), *6*, *48*, *102*
breedbase_refresh, 49
breedbase_refresh(), *110*
breedbase_statistics, 50
breedbase_statistics(), *8*, *43*, *104, 105*, *108*
breedbase_trial_plots, 51
breedbase_trial_plots(), *64*
breedtools *(*breedtools-package*)*, 4
breedtools-package, 4

calculate_environment_stratification_anova,

51
check_design, 52
check_design(), 6, 53–56, 60
check_reset, 53
check_reset(), 54
check_result, 53
check_result(), 6, 53–56
check_section, 54
check_section(), 52, 54
check_setup, 55
check_setup(), 6
check_summary, 56
check_summary(), 6, 52–55
compare_first_stage, 56
compare_first_stage(), 6, 28
complete_environment_info, 57

design_options, 58
design_options(), 5
detect_design, 61

environment_summary_by_group, 62

find_outliers, 63
find_outliers(), 6, 79, 80, 91, 92
first_stage_values, 64
first_stage_values(), 6, 7, 12, 13, 28, 51, 56, 103, 104, 107

get_compatible_groups, 65
get_compatible_groups(), 66
get_maximal_groups, 66
graphics::hist(), 80
guess_column, 66

hartley_critical_sim, 67
hartley_critical_sim(), 68
hartley_fmax_test, 68
hartley_fmax_test(), 6

leaflet::addProviderTiles(), 73
lin_error_mse, 68
lin_group_environments, 69
lin_group_environments(), 6
lin_ss_ge, 70
lin_test_group, 71

make_environment_groups, 71
make_environment_groups(), 65
map_environment_groups, 72
map_environment_groups(), 6

phenotype_template, 74
phenotype_template(), 6, 12, 30, 31, 84, 109, 110
plot_augmented_prep_design, 75
plot_augmented_prep_design(), 5, 35
plot_augmented_row_column_design, 77
plot_augmented_row_column_design(), 5, 79
plot_resolvable_row_column_design, 78
plot_resolvable_row_column_design(), 5, 94
plot_trait, 79
plot_trait(), 6, 48
prediction_accuracy, 80
prediction_accuracy(), 6
prepare_breedbase_pheno, 81
prepare_breedbase_pheno(), 6, 22
prepare_breedbase_trial, 82
prepare_breedbase_trial(), 6, 10, 12, 30, 74, 100
prepare_design_factors, 84
prepare_environment_stratification_data, 85
print.augmented_prep_analysis, 86
print.augmented_prep_two_stage, 86
print.augmented_row_column_analysis, 87
print.resolvable_row_column_analysis, 87
print.single_stage_analysis, 88
print.two_stage_analysis, 88

read_breedbase_file, 89
read_breedbase_file(), 6, 11
read_locations, 90
read_locations(), 6
remove_outliers, 91
remove_outliers(), 6, 48, 110
resolvable_row_column_design, 92
resolvable_row_column_design(), 5, 10, 12, 24, 45, 52, 61, 74, 79, 95, 100
resolvable_row_column_efficiency, 95
resolvable_row_column_efficiency(), 5, 94
run_breedbase_environment_stratification, 96
run_breedbase_variance_check, 97

run_breedbase_variance_check(), *6*
run_environment_stratification, 98
run_environment_stratification(), *6*

save_breedbase, 100
save_breedbase(), *6*, *10–12*, *30, 31*, *34*, *40*, *45–48*, *64*, *74*, *83, 84*, *94*, *102*, *106*, *108, 109*, *111*
save_breedbase_analysis, 103
save_breedbase_analysis(), *6–8*, *13*, *44*, *112*
save_breedbase_coordinates, 106
save_breedbase_coordinates(), *11*
save_breedbase_first_stage, 107
save_breedbase_first_stage(), *6*, *49*, *64*
save_breedbase_phenotypes, 109
save_breedbase_phenotypes(), *6*, *49*, *74*, *107, 108*
second_stage_values, 111
second_stage_values(), *6, 7*, *13*, *103, 104*